

CwellEEGRead

Stakeholder needs, requirements, design and test specifications

Generated from the Doorstop tree in `docs/traceability/`

Tree date	2026-09-29
Repository revision	4b0c964
Needs	7 items (NEED)
Requirements	22 items (REQ)
Design	22 items (DES)
Test specifications	18 items (TST)

Contents

Needs (NEED)	4
1.0 Research access to clinical Cadwell EEG (NEED001)	4
2.0 A standard interchange format for downstream analysis (NEED002)	4
3.0 Trustworthy conversion (NEED003)	4
4.0 Reproducible, traceable, LLM-assisted development (NEED004)	4
5.0 Protection of patient data (NEED005)	4
6.0 Downstream uses are supported, not just enabled (NEED006)	5
7.0 Open-source licensing that permits reuse of prior work (NEED007)	5
Requirements (REQ)	6
1.0 Input: Cadwell EEG recordings from around 2020 onward (REQ001)	6
2.0 Output: EDF/EDF+ files (REQ002)	6
3.0 Signal fidelity (REQ003)	6
4.0 Recording and patient metadata (REQ004)	6
5.0 Events and annotations (REQ005)	7
6.0 Command-line interface (REQ006)	7
7.0 Test data set supplied out of band (REQ007)	7
8.0 Proven equivalence with the native EDF export (REQ008)	7
9.0 Proven equivalence with the native CSV/text export (REQ009)	7
10.0 Round-trip self-consistency (REQ010)	8
11.0 Automated test execution (REQ011)	8
12.0 LLM session provenance (REQ012)	8
13.0 Requirements traceability (REQ013)	8
14.0 Anonymisation option (REQ014)	8
15.0 Licence compatibility (REQ015)	8
16.0 Downstream use scaffolds (REQ016)	9
17.0 Reproducible development environment (REQ017)	9
18.0 Clear failure on unsupported input (REQ018)	9
19.0 Recording gaps and discontinuities (REQ019)	9
20.0 Sample clock and resampling policy (REQ020)	9
20.1 Event placement on the sample clock (REQ021)	10
20.2 EEGLAB plugin import dialog (REQ022)	10
Design (DES)	11
1.0 Input: Cadwell EEG recordings from around 2020 onward (DES001)	11
2.0 Output: EDF/EDF+ files (DES002)	11
3.0 Signal fidelity (DES003)	13
4.0 Recording and patient metadata (DES004)	13
5.0 Events and annotations (DES005)	14
6.0 Command-line interface (DES006)	15
7.0 Test data set supplied out of band (DES007)	16
8.0 Proven equivalence with the native EDF export (DES008)	16

9.0 Proven equivalence with the native CSV/text export (DES009).....	17
10.0 Round-trip self-consistency (DES010).....	18
11.0 Automated test execution (DES011).....	19
12.0 LLM session provenance (DES012).....	19
13.0 Requirements traceability (DES013).....	20
14.0 Anonymisation option (DES014)	22
15.0 Licence compatibility (DES015).....	22
16.0 Downstream use scaffolds (DES016).....	23
17.0 Reproducible development environment (DES017).....	25
18.0 Clear failure on unsupported input (DES018)	26
19.0 Recording gaps and discontinuities (DES019).....	27
20.0 Sample clock and resampling policy (DES020).....	28
21.0 Event placement on the sample clock (DES021)	29
22.0 EEGLAB plugin import dialog (DES022)	29
Test specifications (TST)	31
1.0 EDF structural validity (TST001).....	31
2.0 Header equivalence with native EDF export (TST002)	31
3.0 Sample equivalence with native EDF export (TST003).....	31
4.0 Annotation equivalence with native EDF export (TST004).....	31
5.0 Sample equivalence with native CSV/text export (TST005)	31
6.0 Command-line behaviour (TST006)	32
7.0 Anonymisation (TST007)	32
8.0 Test data manifest and graceful skip (TST008).....	32
9.0 Traceability validation (TST009)	32
10.0 LLM log integrity (TST010)	33
11.0 Environment setup (TST011)	33
12.0 Licence compatibility check (TST012)	33
13.0 Downstream use scaffolds present (TST013).....	33
14.0 Supported-version coverage (TST014)	33
15.0 Gap handling (TST015).....	34
16.0 Event timing on the sample clock (TST016).....	34
17.0 Import through the EEGLAB menu (TST017).....	34
18.0 EEGLAB plugin screenshots, manually verified (TST018).....	34
Traceability matrix.....	36

Needs (NEED)

Why the project exists: the stakeholder needs every requirement traces back to.

1.0 Research access to clinical Cadwell EEG

NEED001

Clinicians and researchers need to analyse EEG recordings made on Cadwell systems (Cadwell Arc / Easy III and later, i.e. recordings from around 2020 onward) for research purposes, without depending on the proprietary Cadwell application. Many research EEGs are recorded for clinical purposes first, so the clinical archive is the source of research data.

Level	1.0	Parents	-	Children	REQ001 , REQ006
-------	-----	---------	---	----------	---

2.0 A standard interchange format for downstream analysis

NEED002

Downstream analysis tools - the SCORE-AI automated EEG interpretation system, the Mergoth EEG foundation model, EEGLAB, FieldTrip, MNE-Python and others - consume EDF/EDF+ files. Converting the proprietary Cadwell format to EDF is therefore the upstream enabler for every intended research use.

Level	2.0	Parents	-	Children	REQ002 , REQ005 , REQ020 , REQ021
-------	-----	---------	---	----------	--

3.0 Trustworthy conversion

NEED003

Research results must not be biased by conversion errors. The converted data must be demonstrably equivalent to what the vendor's own application exports, and this equivalence must be proven by automated tests rather than asserted.

Level	3.0	Parents	-	Children	REQ003 , REQ004 , REQ005 , REQ007 , REQ008 , REQ009 , REQ010 , REQ018 , REQ019 , REQ020 , REQ021
-------	-----	---------	---	----------	--

4.0 Reproducible, traceable, LLM-assisted development

NEED004

The software is developed with large-language-model assistance. Every prompt and session transcript must be archived in the repository, and requirements, design decisions and tests must be traceable to each other, so that a reader without access to an LLM can audit how and why the software came to be.

Level	4.0	Parents	-	Children	REQ011 , REQ012 , REQ013 , REQ017
-------	-----	---------	---	----------	--

5.0 Protection of patient data

NEED005

The test recordings are clinical EEGs of real patients. They must never be committed to the repository, and the converter must offer a way to strip or pseudonymise identifying information when producing research copies.

Level	5.0	Parents	-	Children	REQ004 , REQ007 , REQ014
-------	-----	---------	---	----------	--

6.0 Downstream uses are supported, not just enabled

NEED006

Beyond producing EDF files, the project should make it easy to actually run the intended downstream analyses: scripts to check out and run the Mergoth foundation model, a wrapper to call a SCORE-AI command-line program on the converted files, and an EEGLAB (and possibly FieldTrip) reader plugin.

Level	6.0	Parents	-	Children	REQ006 , REQ016 , REQ022
-------	-----	---------	---	----------	--

7.0 Open-source licensing that permits reuse of prior work

NEED007

Reading the inner Cadwell data format may require porting code from the BioSig toolbox (GPL-licensed). The project's licence must allow that reuse while remaining usable by the research community.

Level	7.0	Parents	-	Children	REQ015
-------	-----	---------	---	----------	------------------------

Requirements (REQ)

What the software shall do. Each requirement links to the needs it serves and is implemented by one design item.

1.0 Input: Cadwell EEG recordings from around 2020 onward

REQ001

The program shall read EEG recordings produced by Cadwell Arc EEG software (recordings from around 2020 onward). A recording is a "CadLink" study export: a folder holding, under `CadLink/Data/`, a set of SQLite 3 databases per record - `<record>-<timestamp>.ezdataindex` (frame index and track definition), `<record>-<timestamp>-<n>.ezdata` (EEG waveform frames), `<record>-<timestamp>.ezevents` (events), `<record>.mediadb` and `<record>-<n>.mediadb` (audio/video frames) - plus encrypted catalogue databases under `CadLink/Databases/` that the program shall not need. The outer structure is readable with a standard SQLite library; the inner waveform encoding is proprietary. The set of supported Cadwell software versions and storage schema versions (SchemaUpdateLog) shall be documented and each supported version shall be represented by at least one test recording.

Level	1.0	Parents	NEED001	Children	DES001
-------	-----	---------	---------	----------	--------

2.0 Output: EDF/EDF+ files

REQ002

The program shall write European Data Format files conforming to the EDF specification (Kemp et al. 1992) and, when annotations, a sub-second start time or a discontinuous time axis (EDF+D) require it, to the EDF+ specification (Kemp & Olivan 2003). Output files shall be readable by at least two independent EDF readers (e.g. EDFbrowser and pyedflib or MNE-Python) without warnings about header validity. Where a common reader cannot represent a file correctly (EDF+D), the documentation shall say so. When the program applies a filter to the signal, the EDF prefilter field shall say so.

Level	2.0	Parents	NEED002	Children	DES002
-------	-----	---------	---------	----------	--------

3.0 Signal fidelity

REQ003

For every EEG channel in the source recording the program shall preserve: the channel label, the sampling rate, the physical dimension (microvolts), the physical and digital minimum/maximum, and every sample value. No filtering, resampling, re-referencing or truncation shall be applied unless explicitly requested on the command line, and any such processing shall be recorded in the EDF header's prefiltering field.

Level	3.0	Parents	NEED003	Children	DES003
-------	-----	---------	---------	----------	--------

4.0 Recording and patient metadata

REQ004

The program shall populate the EDF header with the recording start date and time (frame-0 time stamp corrected by the amplifier/PC clock offset, to the microsecond via the EDF+ sub-second convention; UTC unless a time zone is given), the patient identification fields, the recording identification fields (record identifier, equipment) and per-channel transducer and prefiltering fields. Channel labels shall follow the EDF+ label convention (e.g. EEG Fp1-Cz). Because the Cadwell files store no channel labels, the program shall derive them from a documented amplifier-input layout table (cwellleegread/layout.py, verified on the test recordings), shall allow the user to override the table, and shall flag inferred labels in the conversion report.

Level	4.0	Parents	NEED003, NEED005	Children	DES004
-------	-----	---------	------------------	----------	--------

5.0 Events and annotations

REQ005

Events stored in the Cadwell recording (technician comments, seizure or event markers, photic stimulation and hyperventilation markers, montage or impedance events) shall be exported as EDF+ annotations with onset times accurate to one sample, and with their text preserved. Event types that cannot be mapped shall be listed in the conversion report.

Level	5.0	Parents	NEED002, NEED003	Children	DES005
-------	-----	---------	------------------	----------	--------

6.0 Command-line interface

REQ006

The program shall provide a command-line interface of the form `cwelleegread convert <cadwell-input> <output.edf>` that converts a single recording, plus a batch mode that converts every recording under a folder. It shall return exit code 0 on success and non-zero on any failure, and shall optionally emit a machine-readable JSON conversion report (input identity, channels, duration, events, warnings) for use by downstream scripts.

Level	6.0	Parents	NEED001, NEED006	Children	DES006
-------	-----	---------	------------------	----------	--------

7.0 Test data set supplied out of band

REQ007

Development shall use a small set of test EEGs, each consisting of (a) the recording in native Cadwell format, (b) the same recording exported to EDF by the native Cadwell application and (c) the same recording exported to CSV/text by the native Cadwell application. Recordings of real patients shall be supplied outside the repository and live only under `testdata/private/` (gitignored). Recordings that contain no patient data (synthetic, amplifier noise or consenting volunteers recorded under a placeholder name) may be committed under `testdata/public/`, so that the equivalence tests run on every clone. Every test recording, public or private, shall be listed in `testdata/manifest.json` with its file names, sizes, SHA-256 checksums, storage schema version and the Cadwell software version that produced it, and the checksums shall be verified by the tests.

Level	7.0	Parents	NEED003, NEED005	Children	DES007
-------	-----	---------	------------------	----------	--------

8.0 Proven equivalence with the native EDF export

REQ008

An automated test shall prove that the program's EDF export is equivalent to the native Cadwell EDF export of the same recording: (1) the same set of channels, matched by label after the documented mapping; (2) identical sampling rate per channel; (3) identical recording start time; (4) the same set of annotations (onset, duration, text); (5) sample values equal within one digital quantisation step of the native export. The native EDF export is not the raw data: it resamples surplus samples (Apollo, 250 Hz), pads gaps with zeros, applies a 2nd-order 0.16 Hz Butterworth high-pass primed on the time-reversed segment start (Essentia recordings), starts at the export dialog's start time and drops the last frame, and applies a documented event policy. The test shall therefore compare the program's vendor-compatible mode (REQ020) sample by sample over the whole file, and the raw-fidelity output only against the vendor's text export (REQ009), which is the raw data. Any tolerance actually used shall be justified in the test's documentation.

Level	8.0	Parents	NEED003	Children	DES008
-------	-----	---------	---------	----------	--------

9.0 Proven equivalence with the native CSV/text export

REQ009

An automated test shall prove that the physical sample values in the program's EDF export are equal, within the precision of the text export, to the values in the native Cadwell CSV/text export of the same recording, channel by channel, over the full common time range. The test shall also verify that the channel names in the text export map one-to-one onto the EDF channel labels.

Level	9.0	Parents	NEED003	Children	DES009
-------	-----	---------	---------	----------	--------

10.0 Round-trip self-consistency

REQ010

The EDF file written by the program shall, when read back by an independent EDF reader, yield the same channel labels, sampling rates, start time and sample values (within EDF's 16-bit quantisation) as the data the program held in memory before writing.

Level	10.0	Parents	NEED003	Children	DES010
-------	------	---------	-------------------------	----------	------------------------

11.0 Automated test execution

REQ011

All equivalence and validity tests shall run under `pytest`. Tests that need the out-of-band test data shall skip with an explicit message when the data is absent, so that the remaining tests can run in continuous integration and on machines without access to patient data. A continuous integration workflow shall run the test suite on every push and pull request.

Level	11.0	Parents	NEED004	Children	DES011
-------	------	---------	-------------------------	----------	------------------------

12.0 LLM session provenance

REQ012

Every Claude Code session that touches the repository shall be archived automatically into `llm-logs/<session_id>/` (full transcript, prompts as CSV, prompts and responses as Markdown) and indexed in `llm-logs/sessions.csv`, without manual steps, so that the complete history of prompts and responses is available in the repository.

Level	12.0	Parents	NEED004	Children	DES012
-------	------	---------	-------------------------	----------	------------------------

13.0 Requirements traceability

REQ013

Requirements shall be managed with Doorstop in `docs/traceability/` as a chain `NEED -> REQ -> DES -> TST`. Every normative requirement shall link to at least one need and shall have a design item (DES) stating how it is implemented; every design item shall be covered by at least one test specification, and `doorstop` validation shall pass with no errors before a release. The published tree (Markdown per document and one PDF of the whole tree with the traceability matrix) shall be kept in the repository.

Level	13.0	Parents	NEED004	Children	DES013
-------	------	---------	-------------------------	----------	------------------------

14.0 Anonymisation option

REQ014

The program shall offer a command-line option that replaces the patient identification fields in the EDF header (name, patient code, birth date, sex) with the values given by the user or with anonymous placeholders, and that removes patient-identifying text from exported annotations, without altering the signal data.

Level	14.0	Parents	NEED005	Children	DES014
-------	------	---------	-------------------------	----------	------------------------

15.0 Licence compatibility

REQ015

The repository's licence shall be compatible with every third-party source ported into it. If code from the BioSig toolbox (GPL-3.0-or-later) is ported, the repository shall be relicensed under a GPL-compatible licence and the decision shall be recorded in `docs/research/` and in the LICENSE file, including attribution of the original authors.

Level	15.0	Parents	NEED007	Children	DES015
-------	------	---------	-------------------------	----------	------------------------

16.0 Downstream use scaffolds

REQ016

The repository shall contain a `uses/` folder with one sub-folder per downstream use: `uses/Morgoth` (scripts to check out, install and run the Morgoth foundation model on converted EDF files), `uses/SCOREAI` (scripts to call an externally supplied SCORE-AI command-line program on converted EDF files and collect its JSON output) and `uses/EEGLAB` (an EEGLAB reader plugin that can be packaged as a zip file). Each sub-folder shall have a README stating its status and prerequisites.

The EEGLAB plugin shall represent recording pauses (REQ019) as events: when the pause is padded with zeros, an event of type `Recording gap` with the pause length as duration at the start of the padding; when the segments are concatenated, a standard EEGLAB `boundary` event with the number of removed samples as duration at the join, with the latencies of all later events reduced by the pause length. The pauses shall also be listed in `EEG.etc.cadwell.gaps` in both modes.

Level	16.0	Parents	NEED006	Children	DES016
-------	------	---------	---------	----------	--------

17.0 Reproducible development environment

REQ017

All Python tooling shall be installed into a gitignored virtual environment from pinned requirement files by a single `setup.sh` script, so that a fresh checkout can be brought to a working state with one command.

Level	17.0	Parents	NEED004	Children	DES017
-------	------	---------	---------	----------	--------

18.0 Clear failure on unsupported input

REQ018

When given a recording it cannot fully convert (an unsupported Cadwell version, a corrupt or truncated database, an unknown waveform encoding), the program shall stop with a diagnostic message naming the problem and a non-zero exit code, and shall not leave a partial EDF file that could be mistaken for a complete conversion.

Level	18.0	Parents	NEED003	Children	DES018
-------	------	---------	---------	----------	--------

19.0 Recording gaps and discontinuities

REQ019

Cadwell recordings can contain gaps (recording stopped and restarted); the index then lacks the frame numbers of the gap and holds a GapInfo row. The program shall detect gaps from the frame numbering and represent them in the EDF output, by default as the vendor's own export does: a continuous EDF+C file in which the missing seconds are written as digital zero at the frame boundaries so that the time axis and every annotation stay aligned with wall-clock time, plus an annotation "Recording gap N s (padded with zeros)" at the gap start. This default is chosen because common EDF readers (EDFlib/pyedflib, MNE-Python) do not handle discontinuous files correctly. As an option outside the vendor-compatible mode, the program shall write a discontinuous EDF+D file whose data records carry their true onsets, so that the gap seconds are left out, with an annotation "Recording gap N s". The gaps shall be listed in the conversion report, and the equivalence tests shall compare segment by segment so that gap handling differences are explicit rather than hidden.

Level	19.0	Parents	NEED003	Children	DES019
-------	------	---------	---------	----------	--------

20.0 Sample clock and resampling policy

REQ020

The Cadwell amplifier delivers a variable number of samples per one-second frame (248, 250 or 251 observed at a nominal 250 Hz; exactly 500 at 500 Hz), i.e. its sample clock can run about 0.08 % fast relative to the frame time stamps. By default the program shall preserve every raw sample unchanged ("raw fidelity": no filtering, no resampling, the nominal sampling rate in the EDF header, only a trailing partial second dropped) and report the effective rate and drift in the conversion report. A command-line option shall alternatively reproduce the vendor's EDF export ("vendor-compatible"): all frames but the last, surplus samples removed at evenly spaced positions with two-point smoothing,

gaps and (optionally) the leading missing frames padded with digital zero, the vendor's 2nd-order 0.16 Hz Butterworth high-pass primed on the time-reversed segment start where the vendor applies it (Essentia headbox; overridable), the vendor's physical range and event policy - so that the output matches the native EDF export sample by sample. The chosen policy shall be recorded in the EDF header's recording-additional field and in the conversion report.

Level	20.0	Parents	NEED002, NEED003	Children	DES020
-------	------	---------	------------------	----------	--------

20.1 Event placement on the sample clock

REQ021

Cadwell events carry two times: a wall-clock stamp (`StartTime` , 100 ns resolution) and an offset on the amplifier sample clock (`StartOffset` ticks from the record origin). The stored frames carry the same pair, and the two clocks drift relative to each other (about 96 ppm on the Essentia recordings seen, 0.35 s per hour). The samples are on the tick clock.

The program shall by default place events on the sample clock: the onset of an event is derived from its tick offset through the tick spans of the stored frames (linear within a frame, nominal rate across padded gaps and beyond the stored range), so that an event lands on the sample it belongs to regardless of recording length. Placement by wall-clock stamp, which the vendor's EDF export uses and which lands early by the accumulated drift (the stamp clock runs behind the sample clock), shall remain available as an option (`--event-timing stamp`) and shall be the default in vendor-compatible mode so that the equivalence tests against the vendor's export (REQ008, REQ020) still hold. The conversion report shall state which placement was used. The EEGLAB plugin shall offer the same choice with the same default.

Level	20.1	Parents	NEED002, NEED003	Children	DES021
-------	------	---------	------------------	----------	--------

20.2 EEGLAB plugin import dialog

REQ022

Chosen from EEGLAB's menu, the EEGLAB plugin shall, after the file dialog, show a dialog for its import options: whether to import events, how to represent recording pauses (concatenated with `boundary` events, or padded with zeros, REQ016) and the event placement (sample clock or wall-clock stamps, REQ021), with the command-line defaults preselected; for an EDF file only the event choice. The options chosen shall reach the dataset and be recorded in the EEGLAB history as a complete `pop_cadwell` call. After the import, EEGLAB's standard dataset naming dialog (`pop_newset`) shall follow as for any other importer. Cancel in the options dialog shall create no dataset. Called with a path, the importer shall show no dialog.

Under GNU Octave, the plugin shall supply what EEGLAB's graphical interface needs from Octave and Octave lacks, as far as this can be done without changing EEGLAB: stand-ins that are never on the path under MATLAB, and documented steps for what the plugin cannot do by itself.

Level	20.2	Parents	NEED006	Children	DES022
-------	------	---------	---------	----------	--------

Design (DES)

How each requirement is implemented in the code as it is today: modules, algorithms, constants, decisions and known limits. One design item per requirement.

1.0 Input: Cadwell EEG recordings from around 2020 onward

DES001

Implements REQ001 - the program shall read Cadwell Arc CadLink study exports (SQLite `.ezdataindex`, `.ezdata`, `.ezevents`, recordings from about 2020 on) without the encrypted catalogue databases, and the supported software and schema versions shall be documented and each covered by a test recording.

Design. `cwelleegread/ezdata.py`: `open_recording()` accepts an `.ezdataindex` file or a folder (itself, `native-export/CadLink/Data`, `CadLink/Data` or `Data` below it) and refuses a folder holding more than one record. `CadwellRecording.__init__()` opens the index with the standard `sqlite3` module read-only and immutable (`_ro(): file:...?mode=ro&immutable=1`, so nothing is written to the export) and reads `SchemaUpdateLog` (kept as `schema_versions`), `MediaHeader.MediaDescriptor` (patient, record and author GUIDs), `TrackInfo` track 0 (channel records: channel number, amplifier input, rate), `FrameInfo` track 0 (frame number, UTC stamp, `JoinDatabase`, `FrameKey`), `MiscInfo.AMPLAYOUT` (headbox type), `PcTimeSync` and `GapInfo`. `frames()` groups the index rows by `JoinDatabase`, loads every blob of each named `<n>.ezdata` file keyed by `FrameKey`, and yields frames in index order through `decode_frame(): magic 0x033149BD`, channel blocks located by the tag `ab792193de4225a2`, start/end ticks at `0x32` when the first block sits at `0x42`, delta type 1 (int16) or 2 (int8), `samples = first + cumsum(deltas) x scale`. A missing data file or frame, a bad magic, an unknown delta type or a channel-count mismatch raises an error instead of yielding data. `events()` reads `Events` from the sibling `.ezevents` (empty list if absent). Only track 0 (EEG) is decoded; auxiliary tracks, `.mediadb` files and `CadLink/Databases/` are never opened. Dependencies are the standard library and numpy. The EEGLAB plugin re-implements the same reader (uses `EEGLAB/cadwellio/cadwell_read_index.m`, `cadwell_decode_frame.m`, `cadwell_read.m`) on a pure MATLAB/Octave SQLite file reader (`cadwell_sqlite_native.m`). Porting BioSig was rejected: its Cadwell code decodes nothing and is GPL (`docs/research/biosig-cadwell-reader.md`).

Supported versions. `cwelleegread/ezdata.py`: `SUPPORTED_SCHEMA_VERSIONS` (now `("2.5",)`) lists the storage schema versions verified against a test recording; `CadwellRecording.schema_version` is the `NewVersion` of the last `SchemaUpdateLog` row and `check_supported()` raises `UnsupportedVersionError` (a `ValueError`) for any other. `convert()` calls it first, so an unknown version is refused by default; `--allow-unsupported` converts anyway with a report warning (DES018). `inspect` still opens any version and prints `schema <v> (supported|NOT SUPPORTED)`, and the JSON report carries `schema_version`. The table of supported versions and their test recordings is `testdata/manifest.json` (`schema_version`, `cadwell_software_version` per recording) and the README.

Limits. The three public recordings (four exports: Apollo 250 Hz, Arc version not recorded; Essentia 500 Hz, Arc 3.2.1097) all carry the schema chain 1.0 -> 2.5 and a single `-1.ezdata` file, so other schema versions and multi-file records are untested and refused. The Cadwell software version is not stored in the files the converter reads, so the check is on the storage schema only. The EEGLAB plugin does not check the version.

Verified by TST014 - `tests/test_manifest.py`: `test_every_supported_version_has_a_test_recording` checks that `SUPPORTED_SCHEMA_VERSIONS` is non-empty, that every version has a manifest recording and that each public recording's data carries the version the manifest states; the refusal is tested by TST006 (`tests/test_cli.py`). The decoder itself is exercised by TST005 (DES009).

Level	1.0	Parents	REQ001	Children	TST014
-------	-----	---------	--------	----------	--------

2.0 Output: EDF/EDF+ files

DES002

Implements REQ002 - output files shall conform to EDF (Kemp et al. 1992) and, when annotations are present, EDF+ (Kemp & Olivan 2003), and be readable without header warnings by at least two independent EDF readers.

Design. `cwelleegread/edfwrite.py: write_edf()` is an in-house EDF, EDF+C and EDF+D writer (`write_edf_plus()` is its EDF+C shorthand); `cwelleegread/edf.py: convert()` prepares signals, signal headers and annotations and calls it. A library writer (`pyedflib/EDFlib`) was not used so that annotation texts are not truncated, the sub-second start is written as the first TAL of every record (as the Cadwell export does) and every header field is under the project's control (module docstring). Rules: record duration 1 s; each signal has $\text{round}(\text{rate} \times 1 \text{ s})$ samples per record and all signals must span the same whole number of records (else `ValueError`); physical values are digitised as $\text{rint}((x - \text{pmin}) / (\text{pmax} - \text{pmin}) \times (\text{dmax} - \text{dmin}) + \text{dmin})$, clipped and stored as little-endian `int16`. Header: version 0; EDF+ patient field code sex birthdate name and recording field Startdate dd-`MMM`-`yyyy` admincode technician equipment [additional], spaces replaced by `_` and empty subfields written `X`; start date `dd.mm.yy` and time `hh.mm.ss` in whole seconds; reserved field EDF+C; a final EDF Annotations signal (physical -1..1, digital -32768..32767) sized to the longest record's TAL list, at least 30 samples. Every record's annotation block starts with the time-keeping TAL `+<subsecond + k>`; event TALs carry the onset plus the sub-second start (onsets are relative to the header's whole second), an optional duration after `0x15`, and UTF-8 text, and go into the record of their onset (clamped to the first/last record). Fields are ASCII-encoded; a value that does not fit raises `ValueError` instead of being cut (`_field()`, `_num()` picks the shortest numeric form fitting 8 characters); start years outside 1985-2084 are refused.

Which variant (`convert(edf_format=...)`, CLI `--format auto|edf+|edf`). `auto` writes plain EDF when nothing needs EDF+ - no annotations, no gap left out (EDF+D) and a start on a whole second, since plain EDF has no time-keeping TAL for a sub-second start; that happens only with every event filtered out, so almost every conversion is EDF+. Plain EDF has a blank reserved field and no annotation signal; the patient and recording fields keep the EDF+ subfield layout, which is valid free text in EDF. `edf+` always writes EDF+; `edf` forces plain EDF, dropping the annotations and truncating the start to the whole second, each with a report warning. EDF+D (`edf_type='edf+d'`, used for recording gaps with `--gaps discontinuous`, DES019) takes one onset per record; the onsets must increase by at least the record duration; each record's time-keeping TAL carries its onset; and an event goes into the last record starting at or before it, so an event inside a gap is written in the record before the gap. The report states `edf_type`.

Prefilter. When `convert()` applies the vendor high-pass (vendor mode on Essentia, or `--highpass on`), every signal's prefilter field reads `HP: unknown (0.16 Hz?)` (`edf.py: HIGHPASS_PREFILTER`). The vendor documents no filter and leaves the field empty even in its high-passed exports; the 0.16 Hz cut-off is this project's identification, hence the question mark. This is a deliberate difference from the vendor header. Without the filter the field stays empty. The CLI (`cwelleegread/__main__.py: cmd_convert()`) writes `<output>.part` and renames it on success, so a failed conversion leaves no file.

Readers. EDF and EDF+C files read back identically with `pyedflib` (EDFlib) and MNE-Python (1.13). EDF+D is not handled by either: EDFlib/`pyedflib` refuses a truly discontinuous file ("The file is discontinuous and cannot be read"), and MNE-Python reads it without a warning as if contiguous, so after a gap its time axis is early by the gap length while the annotations keep their true onsets. EDFbrowser supports EDF+D. This is why gaps are zero-padded into EDF+C by default and EDF+D is opt-in (`--gaps discontinuous`, DES019, README).

Limits. `convert()` cuts labels to 16 and the recording-additional text to 34 characters. Neither reader used in the tests reports the sub-second start faithfully: MNE-Python keeps whole seconds, and `pyedflib` 0.1.x returns .027665 for a time-keeping TAL of .276647. The tests therefore check the TAL itself. EDFbrowser is not run by the tests.

Verified by TST001 - converts public exports 1 and 2 in raw mode and export 1 in vendor mode and reads them back with `pyedflib`, checking record count, duration, channel count and samples within half the declared resolution (`tests/test_convert_public.py`); `tests/test_edf_formats.py` reads raw and vendor output with MNE-Python and compares it with `pyedflib` within the resolution, and checks plain EDF (automatic and forced), EDF+D record onsets and samples with its own record-level reader, the EDF+D writer's input checks, and the prefilter field with and without the high-pass.

Level	2.0	Parents	REQ002	Children	TST001
-------	-----	---------	------------------------	----------	------------------------

3.0 Signal fidelity

DES003

Implements REQ003 - for every EEG channel the label, sampling rate, microvolt dimension, physical and digital range and every sample value shall be preserved, with no filtering, resampling, re-referencing or truncation unless requested on the command line and then recorded in the EDF prefiltering field.

Design. Fidelity is the default `--mode raw` of `cwelleegread/edf.py: convert()`. `Samples: read_padded()` concatenates the decoded frames of each amplifier input (sorted amplifier-input order, which is the vendor's export column order) and multiplies by `cwelleegread/ezdata.py: UNIT_UV = 0.72998046 μV` per amplifier unit, an empirical constant fitted against the vendor's text export (bounded to [0.729980459, 0.729980461] at 250 Hz, consistent at 500 Hz; no closed form found; `convert(unit_uv=...)` can override it). Nothing is filtered, resampled or re-referenced; data stay referential to the recording reference (Cz, input 15). The only length changes are dropping the trailing partial second (`data[: (n_raw // rate) * rate]`, reported as `samples_dropped_at_end`) and zero-padding recording gaps (REQ019). Rate: the EDF `sample_frequency` is the nominal rate from the channel record (250 or 500 Hz). Dimension: `uV`. Labels: the Cadwell files store no channel labels, so `cwelleegread/layout.py: default_labels()` reconstructs EEG `<name>-<reference>` from a per-headbox table (Apollo type 5, Essentia type 1) chosen by the `AMPLAYOUT` amplifier type; an unknown type falls back to the Essentia table with a report warning, and `--labels` overrides. Ranges: the source has no physical or digital range of its own. Raw mode uses `nice_range()`: a symmetric per-channel range covering the peak plus 1 %, rounded up to two significant digits, at least $\pm 100 \mu V$, over digital -32768..32767, so the step is about $\text{peak}/32767$ - finer than one amplifier unit for ordinary EEG. The vendor's fixed range ($\pm 562500 \mu V$ on Apollo, $17.17 \mu V$ per step) was rejected for raw mode as too coarse (research note) and is kept only in vendor mode. Sample values are therefore preserved within half a quantisation step, not bit for bit.

Deviations found. The prefilter field is always written empty: the 0.16 Hz high-pass (`--highpass on`, also available in raw mode) and vendor-mode resampling are recorded only in the JSON report (`highpass_hz`, `mode`) and, for the mode, in the recording-additional field, contrary to the last sentence of REQ003. Dropping the trailing partial second is not optional (REQ020 permits it). Labels are cut to 16 characters.

Verified by TST002 - compares labels, sampling rates, physical ranges and start time with the vendor EDF (`tests/test_convert_public.py`, vendor mode, export 1; `tests/test_export3_gap.py`, labels and start time, export 3). TST003 - raw fidelity through the text export (TST005) and vendor-mode equivalence of every sample within one quantisation step against the vendor EDFs (`tests/test_convert_public.py`, `tests/test_vendor_exports_filtering.py`).

Level	3.0	Parents	REQ003	Children	TST002, TST003
-------	-----	---------	--------	----------	----------------

4.0 Recording and patient metadata

DES004

Implements REQ004 - the EDF header shall carry the clock-corrected start time (sub-second via EDF+), patient and recording identification, per-channel transducer and prefilter fields, and EDF+ channel labels taken from a documented, overridable amplifier-input table, with inferred labels flagged in the report.

Design.

- *Start time.* `cwelleegread/ezdata.py: CadwellRecording.__init__` sets `start_time` = time stamp of the first stored track-0 frame + (`PcTime` - `SyncTime`) of the first `PcTimeSync` row, the vendor's rule (+341.4 μs on export 1). `parse_timestamp()` truncates the 100 ns stamps to microseconds.
`cwelleegread/edf.py: convert()` converts to the `--timezone` IANA zone (UTC otherwise; no zone is read from the files) and, with `--start-at record-origin`, moves the start back by the padded leading seconds. `cwelleegread/edfwrite.py: write_edf_plus()` writes whole seconds in the header and the sub-second part (6 decimals) as the first TAL of every record, as the vendor does; dates outside 1985-2084 are refused.
- *Identification.* Patient field `<code> X X <name>`: `code` = patient GUID from `MediaHeader.MediaDescriptor`, sex and birth date always X, name from `--patient-name` (default X). The real name, birth date and sex are not read: they sit in the encrypted `EEGRECORDINFO` blob. Recording

field `Startdate dd-MMM-yyyy <record GUID, 8 chars> X CadwellArc`

`CwellEEGRead_<mode>_<zone>`; full GUIDs go to the JSON report. The vendor's technician and equipment strings are not reproduced.

- *Channel fields.* Dimension `uV`, transducer `X`, prefilter empty - also when `--highpass` applies the 0.16 Hz vendor filter, so the header does not record that filter.
- *Labels.* No labels exist in the readable files, so `cwelleegread/layout.py: parse_amp_layout()` reads the amplifier type (`u32` at byte 32 of the `MiscInfo AMPLAYOUT` blob) and `headbox_for()` selects a `HEADBOXES` table (5 = Apollo, 1 = Essentia; taken from the vendor exports and checked physiologically). `default_labels()` builds `EEG <name>-<ref>` (Cz, or the table's `refs`); an unknown type falls back to Essentia (`DEFAULT_HEADBOX`). `--labels (load_labels_file(), amp_input label lines)` overrides entries verbatim; labels are cut to 16 characters. The report always warns that labels are inferred and flags an unknown type. Rejected: the encrypted `EEG.db`; parsing `MontageEvent` trace names is left for later. The EEGLAB plugin uses the same table (`cadwell_layout.m`) and start rule.

Verified by TST002 - header equivalence with the vendor EDF in `tests/test_convert_public.py`

(`test_vendor_mode_reproduces_native_edf`: labels, rates, ranges, start to the second;

`test_vendor_start_subsecond`: sub-second TAL, export 1) and `tests/test_export3_gap.py`:

`test_labels_and_start_time_match_vendor` (Essentia labels, headbox, start); patient and recording fields are not compared.

Level	4.0	Parents	REQ004	Children	TST002
-------	-----	---------	--------	----------	--------

5.0 Events and annotations

DES005

Implements REQ005 - events in the Cadwell recording shall become EDF+ annotations with onsets accurate to one sample and full text, and unmapped types shall be listed in the report.

Design.

- *Reading.* `cwelleegread/ezdata.py: CadwellRecording.events()` reads the `Events` table of `<record>.ezevents` (type, text, wall-clock `StartTime/EndTime`, tick `StartOffset/EndOffset`, `Deleted`, `Priority`) into `Event` objects in time order; no file gives no events.
- *Selection.* `cwelleegread/edf.py: convert()` applies the vendor's policy (research note, "What the vendor's exports filter", rule 7): deleted events, the bookkeeping types in `SKIPPED_EVENT_TYPES` (`AmpConfigurationData`, `LiveAmpConfigurationData`, `ReviewedDataEvent`, `ContinuousImpedanceEvent`, `BaselineImpedanceEvent`) and the single flashes in `SKIPPED_EVENT_TEXTS` (`Photic Stim`; `Photic Start N Hz` is kept) are omitted; `--all-events` keeps all but deleted ones. Events outside the written seconds are dropped. Every omitted event is listed in the report's `events_skipped` with a reason. There is no type mapping; the annotation is the event's `Text` verbatim, so no type is unmappable, and the type itself is not written. Impedance-measurement types and flash markers, named in REQ005, are therefore exported only with `--all-events`.
- *Onset.* With `--event-timing ticks` (default in raw mode) `event_sample()` maps `StartOffset` through the per-frame tick spans collected by `read_padded()` - linear within a frame, nominal rate across gaps and beyond the ends - so events sit on the sample clock; `duration = tick difference`. With `stamp` (default in `--mode vendor`) `onset = StartTime` minus the first frame's stamp, reproducing the vendor, early by the clock drift (about 96 ppm, 0.35 s/h on Essentia). One-sample accuracy therefore holds for `ticks` only (REQ021).
- *Writing.* `cwelleegread/edfwrite.py: write_edf_plus()` places each annotation in the TAL of its record, adds the sub-second start, writes onsets to 1 μ s, replaces `\x14/\x00` in texts, writes UTF-8 and sizes the annotation signal to the longest record; a library writer was rejected because it truncates texts. Zero durations are written without a duration.
- *Rejected.* `Metadata.json` stamps (no extra precision; local time without zone); stamp placement as default (drift, confirmed by the flash-VEP latency). The EEGLAB plugin (`cadwell_read_events.m`, `pop_cadwell.m: event_table`) uses the same skip list and timing option, keeping the type in `cadwelltype`, but has no `--all-events`.

Verified by TST004 - vendor-mode annotations equal the vendor EDF's to the ms in `tests/test_convert_public.py` (export 1) and `tests/test_vendor_exports_filtering.py` (exports 2, 3-withfilter); the raw-mode subset check is not implemented. TST016 - ticks and stamp onsets of the photic flashes differ by the frame drift and the flash VEP latency confirms the tick axis, in `tests/test_event_timing.py` and `cadwell_selftest.m` check F.

Level	5.0	Parents	REQ005	Children	TST004, TST016
-------	-----	---------	--------	----------	----------------

6.0 Command-line interface

DES006

Implements REQ006 - a command line `convert <cadwell-input> <output.edf>` for one recording, a batch mode for a folder, exit code 0 on success and non-zero on failure, and an optional JSON conversion report.

Design. `cwelleegread/__main__.py: main()` is an `argparse` parser (prog `cwelleegread`, `--version`) with three sub-commands. `pyproject.toml` declares the console script `cwelleegread = cwelleegread.__main__:main`; `setup.sh` installs the package editable (`pip install -e .`), so `.venv/bin/cwelleegread` exists, and `python -m cwelleegread` works as before. `inspect <input>` prints record, schema version (with supported or NOT SUPPORTED), start, rate, channels, frames, gaps and the first `--max-events` events. `convert <input> <output>` (`cmd_convert()`) resolves the input with `cwelleegread/ezdata.py: open_recording()` (an `.ezdataindex` file, a `CadLink/Data` folder or an export folder; a folder holding several records is refused) and passes `--mode raw|vendor`, `--timezone`, `--labels`, `--anonymize`, `--patient-name`, `--all-events`, `--highpass auto|on|off`, `--start-at first-frame|record-origin`, `--event-timing auto|ticks|stamp`, `--gaps pad|discontinuous`, `--format auto|edf+|edf` and `--allow-unsupported` to `cwelleegread/edf.py: convert()` through `convert_one()`. The options are shared with batch mode (`add_convert_options()`).

Batch mode. `batch <folder> <outdir>` (`cmd_batch()`) finds every `.ezdataindex` below the folder (recursive, sorted) and converts each with the same options into `<outdir>/<record GUID>-<timestamp>.edf` (the index file's name), adding `-2`, `-3` ... when the same record occurs twice (e.g. two exports of one recording). A failing recording is reported (`error: <path>: <message>` on stderr) and the batch goes on. An existing output is refused unless `--force`, per file. `--reports` writes each conversion report next to its EDF, and `--json` writes a summary: input folder, counts, and per recording input, output, status, error or seconds, EDF type and warnings. Exit code 0 only if every recording converted; 1 if any failed or none was found.

Failure handling: an existing output is refused unless `--force`. The EDF is written to `<output>.part` and moved into place with `os.replace()` only after `convert()` returns; on any exception the `.part` file is removed, so a failed conversion leaves no output. `main()` turns `FileNotFoundError`, `KeyError`, `ValueError` (including `UnsupportedVersionError`) and `sqlite3.DatabaseError` (a corrupt or truncated file) into one `error: ...` line on stderr and exit code 1, and any other exception into a traceback and exit code 1; `argparse` usage errors exit with 2. A closed output pipe (`inspect ... | head`) ends quietly with 0. Success prints a one-line summary and every report warning, and returns 0.

JSON report: `convert()` always builds a report dict and `--json <path>` writes it. It holds the program version, EDF type (EDF, EDF+C, EDF+D), gap policy, records written and omitted, schema version, input and output paths, record and patient GUIDs (patient GUID omitted with `--anonymize`), schema versions, mode, high-pass, start and event-timing policy, start time (UTC and as written), clock correction, headbox, nominal and effective rate, frame and sample counts, seconds, gaps, per-channel label, range and resolution, `annotations_written`, `events_skipped` with reasons, and warnings. The annotations themselves are not listed.

Limits: the JSON is written after the EDF is moved into place, so a failure writing the report exits 1 but leaves the EDF behind. Batch mode converts one recording at a time; there is no parallelism and no resume beyond `--force` being off by default.

Verified by TST006 - `tests/test_convert_public.py: test_cli_behaviour()` runs the CLI as a subprocess on public export 1 (exit 0, EDF written, JSON `seconds == 45`), on a non-existent path (non-zero, `error` on stderr, no output file) and onto an existing output without `--force` (non-zero). `tests/test_cli.py` covers an unsupported schema (refused with a one-line diagnostic and no output, `inspect` still works, `--allow-unsupported` converts with a warning), a truncated index (one-line diagnostic, no output), batch mode (two copies of one record and one

unsupported record: two EDFs named `<stem>.edf` and `<stem>-2.edf`, per-file reports, a summary with the failure, exit 1, overwrite refused without `--force`, no recordings found) and the installed `cwelleegread` command with `inspect | head`.

Level	6.0	Parents	REQ006	Children	TST006
-------	-----	---------	--------	----------	--------

7.0 Test data set supplied out of band

DES007

Implements REQ007 - test recordings, each with native Cadwell files, the vendor's EDF export and the vendor's CSV/text export; patient recordings only under the gitignored `testdata/private/`, non-patient recordings allowed under `testdata/public/`; every recording listed in `testdata/manifest.json` with names, sizes, SHA-256, schema and Cadwell software version, and the checksums verified by the tests.

Design. `.gitignore` excludes `testdata/private/`. `testdata/README.md` defines `testdata/manifest.json`: a recordings list whose entries carry `id`, `location` (`public` or `private`, default `private`), `cadwell_software_version`, `schema_version`, `notes` and three file entries `cadwell`, `native_edf`, `native_csv`, each `{file, sha256, bytes}` relative to `testdata/<location>/`, and asks for the export settings to be written down. `testdata/make_manifest.py` fills in only `sha256` and `bytes` (a study folder is hashed over its sorted relative paths and contents) and skips entries whose files are absent. `tests/conftest.py` loads that script's `sha256_of()` and provides the fixtures `manifest` (skips when the manifest is missing or empty) and `testdata` (skips when the manifest lists no private recording or `testdata/private/` is absent, otherwise asserts that every private file exists and matches its SHA-256).

The manifest lists the four committed public exports (about 300 MB under `testdata/public/`), which the equivalence tests use. They are Cadwell Arc recordings of no real patient, each with a README describing the export dialogs:

- `cadwell-export1`: Apollo headbox, 250 Hz, 46 s of amplifier noise; study export, vendor EDF+ and a 30 s text export.
- `cadwell-export2`: Essentia, 500 Hz, 16 min volunteer EEG with hyperventilation and photic stimulation; vendor EDF from frame 1 and a 30 s text export.
- `cadwell-export3`: Essentia, 500 Hz, 20 min with a 10 s recording break; vendor EDF starting at frame 30 and a full-length text export (zip, 603000 rows).
- `cadwell-export3-withfilter`: byte-identical native files re-exported with a 10-15 Hz viewer filter; EDF from the record origin and a full text export.

Keeping these files private was rejected: they contain no patient data, and committing them makes every equivalence test reproducible on any clone (REQ007 was amended to allow this). Moving them to Git LFS or a release download was considered and not done: it would not shrink the existing history, and CI would need the download. The vendor's "CSV" is its tab-separated text export with a decimal comma (`.txt`; zipped for exports 3).

Limits: export 1's Arc software version was not recorded (manifest: "version not recorded"); exports 2 and 3 are Arc 3.2.1097. No clinical recordings have been supplied under `testdata/private/`, so the private path is exercised only by its skip. The screenshots and READMEs under `testdata/public/` are documentation and are not listed in the manifest.

Verified by TST008 - `tests/test_manifest.py` checks that every manifest entry is complete and that each public recording's three files match their SHA-256 (about 2 s for 300 MB); private entries are checked by `tests/conftest.py: testdata()` when present. TST014 checks the schema versions against the data.

Level	7.0	Parents	REQ007	Children	TST008
-------	-----	---------	--------	----------	--------

8.0 Proven equivalence with the native EDF export

DES008

Implements REQ008 - an automated test proving that the program's EDF equals the native Cadwell EDF export in channels, rates, start time, annotations and every sample within one quantisation step, using the vendor-compatible mode (REQ020).

Design. The native EDF is processed, so the test compares it with `cwellleegread/edf.py`:

`convert(mode="vendor")`. That mode reproduces the export rules identified in `docs/research/cadwell-file-format.md` ("Findings from test export 2", "What the vendor's exports filter"):

1. `vendor_highpass()`: `butter(2, VENDOR_HIGHPASS_HZ = 0.16, "high")` applied causally (`lfilter`) to each contiguous segment. It is primed by first filtering the time-reversed first 20 s, first sample included; padded gap seconds stay zero and priming restarts after each gap. `--highpass auto` enables it in vendor mode on the Essentia headbox only (the Apollo export is unfiltered). Zero-state, steady-state and constant-extension start-ups were rejected: they miss by hundreds to thousands of steps.
2. `vendor_resample()` drops the last frame. It removes the $S = N_{in} - N_{out}$ surplus samples (Apollo frames of 251) with period $T = \text{ceil}(N_{in} / (S + 1))$, and the two output samples before each removal are two-point means. Constant-ratio interpolation was tested and does not match.
3. Gaps are zero-padded at frame boundaries (`read_padded()`); `--start-at record-origin` pads from tick 0.
4. `Start` = the first frame's stamp + the first `PcTimeSync` correction (`CadwellRecording.start_time`); the sub-second goes in each record's first TAL.
5. Annotations use stamp placement. Deleted events, `SKIPPED_EVENT_TYPES` and the `Photic Stim` flashes are dropped.
6. Physical range: Apollo $\pm 562500 \mu\text{V}$; Essentia $\pm 32767 \times 0.72998046 = \pm 23919.27 \mu\text{V}$.

The tests read both files with `pyedflib` and compare record count, array shape and every sample of every channel except column 14 (Cz-Cz); for export 1 also labels, rates and physical ranges. The tolerance is one step of the native file: Apollo $1125000/65535 = 17.17 \mu\text{V} (\times 1.001)$. Essentia is $(23919.03 + 23919.0)/65535 \mu\text{V}$ with 1.05 steps; the extra 0.05 covers the vendor's asymmetric range against our symmetric one (observed maximum 1.04). Annotations are compared as sorted (onset rounded to 1 ms, text).

Limits: annotation durations are not compared. Header start is asserted only for export 1 (for Essentia only the vendor file's start rule is checked). Patient and recording fields differ on purpose and are not compared. A user-chosen start (export 3 at frame 30) is unsupported, so `cadwell3.edf` is checked only spectrally.

Verified by

- TST002 - header fields in `tests/test_convert_public.py`: `test_vendor_mode_reproduces_native_edf` and `test_vendor_start_subsecond`; Essentia labels and start rule in `tests/test_export3_gap.py`: `test_labels_and_start_time_match_vendor`.
- TST003 - all 11000 samples of export 1 (`test_convert_public.py`) and the 480000 / 608500 samples of exports 2 and 3-withfilter (`tests/test_vendor_exports_filtering.py`: `test_vendor_mode_reproduces_essentia_edf_in_full`).
- TST004 - the annotation lists in those same tests.
- TST015 - raw-mode gap padding in `tests/test_export3_gap.py`.

Level	8.0	Parents	REQ008	Children	TST002, TST003, TST004, TST015
-------	-----	---------	--------	----------	--------------------------------

9.0 Proven equivalence with the native CSV/text export

DES009

Implements REQ009 - an automated test proving that the physical sample values equal the vendor's text export within its precision, channel by channel over the full common range, and that the text export's channel names map one to one onto the EDF labels.

Design. The text export is the raw-fidelity reference because it holds the raw data: viewer filters, montage and sensitivity do not reach it (research note, "What the vendor's exports filter", item 1). Each test parses it itself; there is no shared parser module. Lines starting with % (the header) are skipped. Each row is tab-separated: first a 1-second wall-clock stamp, then values in mV with 4 decimals and a decimal comma, converted with `replace(",", ".", "")` and $\times 1000$ to μV . The row count gives the sample index, not the stamps, because Apollo seconds hold 248-251 samples. Columns follow amplifier-input order, which equals `CadwellRecording.amp_inputs`, the same signal order the EDF writer uses with labels from `cwellleegread/layout.py`: `default_labels()`.

Comparison: frames decoded by `cwelleegread/ezdata.py` are multiplied by `UNIT_UV = 0.72998046` $\mu\text{V}/\text{unit}$. The text export omits recording gaps and the last frame, so the decoded frames are concatenated without padding or alignment. For exports 2 and 3-withfilter they start at the frame whose stamp falls in the text's first second (local UTC+2); for export 3 they are frames 1-1216. The arrays must have the same shape. Tolerance: $0.06 \mu\text{V}$. That is $0.05 \mu\text{V}$ (half the $0.1 \mu\text{V}$ text resolution) plus $0.01 \mu\text{V}$ for the empirically fitted scale constant; the 250 Hz and 500 Hz exports bound it to overlapping intervals around 0.72998046.

Coverage: export 1 $7755 \text{ rows} \times 32$ (the text covers 30 of 46 s); export 2 15500×32 (30 s); export 3 and 3-withfilter the whole recording, 603000×32 across the break.

Design decision: the tests compare the decoded samples in memory (`read_signals()`, `frames()`), not values read back from the EDF. EDF quantisation is covered by the round trip (REQ010/DES010), so text = decoded = EDF within resolution holds by transitivity. Comparing the text export with the vendor EDF was rejected because that EDF is filtered and resampled.

Limits: **the one-to-one name mapping is not tested.** The text header's label line is unreliable (export 1 lists 35 labels for 32 columns). The mapping rests on column order = amplifier input, which the sample equality confirms implicitly because a wrong column would fail at $0.06 \mu\text{V}$. The Essentia labels are also checked against the vendor EDF. No test parses the header labels.

Verified by TST005 - `tests/test_ezdata_public.py: test_equivalent_to_text_export` (export 1), `tests/test_vendor_exports_filtering.py: test_text_export_is_raw_even_with_viewer_filters` (exports 2 and 3-withfilter) and `tests/test_export3_gap.py: test_full_text_export_equivalence_at_500hz` (export 3, all 603000 rows).

Level	9.0	Parents	REQ009	Children	TST005
-------	-----	---------	--------	----------	--------

10.0 Round-trip self-consistency

DES010

Implements REQ010 - the EDF the program writes, read back by an independent reader, yields the same labels, sampling rates, start time and samples (within 16-bit quantisation) as the data held in memory before writing.

Design. The file is written by the project's own `cwelleegread/edfwrite.py: write_edf_plus()` (EDF+C, 1 s records, one EDF Annotations signal) and read back in the tests with `pyedflib` (the C library EDFlib), so the check is not self-referential. A library writer was rejected so that annotation texts are not truncated, the sub-second start goes into the first TAL of every record as in the vendor's export, and every header field is under control.

Quantisation: `d = rint((x - pmin) / (pmax - pmin) * (dmax - dmin) + dmin)`, clipped to `[-32768, 32767]`. In raw mode `cwelleegread/edf.py: nice_range()` gives each channel a symmetric physical range: $\max|x| \times 1.01$, at least $100 \mu\text{V}$, rounded up to two significant digits. The step is then about $1/32767$ of the channel's peak and no sample is clipped. Vendor mode uses the vendor's fixed range. Header numbers are written by `_num()` in at most 8 characters. The start is a naive datetime (UTC, or `--timezone`) with whole seconds in the header and the sub-second (6 decimals) in the TALs. Raw mode keeps every sample and drops only a trailing partial second (`samples_dropped_at_end` in the report). The writer raises on overlong fields, signals that do not fill whole records, or a start year outside 1985-2084.

Acceptance rule: every read-back sample is within $0.51 \times \text{resolution}$ of the decoded value (half a step plus floating-point margin). Each channel's resolution must be at most $\max(0.05 \mu\text{V}, 2.3 \times \text{peak}/65535)$, so a data-driven range cannot become coarse.

Limits: only `pyedflib` reads back; MNE-Python, named in TST001, is not used and not a dependency. The in-memory reference is `CadwellRecording.read_signals()`, i.e. the decoded data. In raw mode without `--highpass on` that is exactly what was written. Vendor-mode output is compared with the vendor's file (DES008), not with memory. Labels are asserted on read-back only against the vendor's labels (export 3), and sampling rates only through samples per record.

Verified by TST001:

- `tests/test_convert_public.py: test_raw_mode_round_trip` - export 1: 45 records, 11250×32 samples, 8 samples dropped, per-channel resolution bound, every sample within 0.51 step, start 13:37:50 UTC, 7 annotations.

- `test_export2_converts` (same file) - export 2: 961 records, 480500×32 , 52 annotations, local start time, Cz reference ≈ 0 .
- `tests/test_export3_gap.py` (`raw_edf` fixture) - export 3: 1217 records, padded zeros within one step, labels equal to the vendor's.

Level	10.0	Parents	REQ010	Children	TST001
-------	------	---------	--------	----------	--------

11.0 Automated test execution

DES011

Implements REQ011 - all equivalence and validity tests run under `pytest`, and tests needing the out-of-band data skip with an explicit message when it is absent, so the rest can run in CI and without patient data.

Design. `pytest.ini` sets `testpaths = tests`, `pythonpath = .` (the package is imported without installation) and `addopts = -ra` (the summary lists every skip with its reason), and declares a marker `testdata`. `setup.sh` installs `requirements-dev.txt` (doorstop 3.2, `pytest` ≥ 8 , `pyyaml`, plus the runtime `requirements.txt`: `numpy`, `pyedflib`, `scipy`) into the gitignored `.venv`. Skips, by missing resource:

- Test exports: module-level `pytestmark = pytest.mark.skipif(not (<export>/"native-export").exists(), reason="public test export ... missing")` in `test_convert_public.py`, `test_ezdata_public.py`, `test_export3_gap.py` and `test_vendor_exports_filtering.py`. `test_event_timing.py` and `test_metadata_json.py` call `pytest.skip()` per case.
- Private data: the `tests/conftest.py` fixtures `manifest` and `testdata` skip with a message naming the missing manifest, a manifest without private recordings or a missing `testdata/private/`. No test uses them yet.
- Libraries: `pytest.importorskip("pyedflib")`, `importorskip("scipy.signal")` and `importorskip("mne")`.
- Package: `tests/test_cli.py` skips the console-command check when the package is not installed.
- Tools: `tests/test_octave_port.py` skips without `octave-cli/octave`. `tests/test_eeglab_import.py` also needs `EEGLAB_DIR` (with `functions/`) and `dipfit` (`DIPFIT_DIR` or `<EEGLAB_DIR>/plugins/dipfit`).

The test data are committed public recordings (DES007), so on a full clone the equivalence tests run instead of skipping. The skip paths cover pruned checkouts and missing tools.

Execution: `.github/workflows/tests.yml` runs on every push (branches), pull request and manual dispatch, cancelling superseded runs of the same ref. Job `python` (Python 3.11 and 3.12) checks out the repository with the public test data, runs `./setup.sh` as on a fresh clone (which also validates the Doorstop tree), checks the Doorstop pin, the ignored `.venv` and the `cwellleegread` command, then runs the whole `pytest` suite; the Octave tests skip there. Job `octave` installs GNU Octave, zip and unzip, sparse-clones `EEGLAB functions/` and `dipfit` into `.cache/` as the SessionStart hook does, and runs `tests/test_octave_port.py` and `tests/test_eeglab_import.py`. `release-cadwellio.yml` still builds the plugin zip on a `cadwellio-v*` tag. On Claude Code on the web the SessionStart hook `.claude/hooks/session-start.sh` prepares the same environment for interactive runs.

Limits: the `testdata` marker is declared but never applied. The CI checkout carries the 300 MB of public test data (no LFS). `EEGLAB` and `dipfit` are cloned at their current heads, not pinned. The Octave self-test has a 30-minute timeout.

Verified by TST008 - the public manifest checksums are verified by `tests/test_manifest.py`; the skip when `testdata/private/` is absent is the `tests/conftest.py` fixture, not yet requested by a test. TST011 covers the CI job that runs the suite on a fresh clone.

Level	11.0	Parents	REQ011	Children	TST008
-------	------	---------	--------	----------	--------

12.0 LLM session provenance

DES012

Implements REQ012 - every Claude Code session shall be archived into `llm-logs/<session_id>/` automatically (transcript, prompts as CSV, prompts and responses as Markdown) and indexed in `llm-logs/sessions.csv`, with no manual step.

Design. Four Claude Code hooks call one stdlib-only script.

- **Hooks.** `.claude/settings.json` registers four hooks. `SessionStart` runs `.claude/hooks/llmlog-session-start.sh`, `UserPromptSubmit` runs `llmlog-prompt.sh`, `Stop` (after every assistant turn) runs `llmlog-archive.sh`, and `SessionEnd` runs `llmlog-archive.sh` and then `llmlog-commit.sh`. Each wrapper finds the repository root (`$CLAUDE_PROJECT_DIR` or `git rev-parse`) and `python3`, then `execs llm-logs/tools/llmlog.py <sub-command>`. If either is missing, it exits 0.
- **Payload.** `llmlog.py: read_payload()` reads the hook JSON from stdin. If the JSON has no session id it uses `CLAUDE_CODE_SESSION_ID`. If it has no transcript path it uses the default location `~/.claude/projects/<mangled root>/<id>.jsonl`.
- **Index.** `cmd_session_start()` and `upsert_session_row()` add or update a row in `sessions.csv`. The row holds the start time, branch, user (`PROMPT_LOG_USER`, `CLAUDE_CODE_USER_EMAIL` or the OS user), source and topic, which starts as `untagged`. The `topic` column is the only field meant to be edited by hand.
- **Real-time prompt log.** `cmd_prompt()` appends each human prompt to `llm-logs/prompt-log.csv`. It skips system-injected turns (text starting with `<`, `Caveat:` or `[Request interrupted]`). (The docstring says it also writes `prompts.csv`; the code does not.)
- **Archive.** `cmd_archive()` copies the transcript to `<session_id>/transcript.jsonl`. A session resumed in a fresh container produces a smaller, compacted transcript. If the archived copy is larger than the incoming transcript, the new copy goes to `transcript.resumed.jsonl` instead, so the larger archive is never overwritten. `write_index()` then regenerates, per session folder (`write_session_files()`, human prompts found by `is_human_prompt()`), `session.json`, `prompts.csv` and `PROMPTS-AND-RESPONSES.md` and back-fills `prompt-log.csv`. It also rewrites `SESSIONS.md`. Finally it runs `git add llm-logs`.
- **Commit.** `cmd_commit()` runs `git commit -o llm-logs` with the message `llm-logs: archive session log (automatic)` and pushes to the current branch on `origin`. The commit contains only `llm-logs/`, never other work.
- **Never blocking.** `main()` catches every exception, writes `llmlog: ...` to stderr and exits 0, so a logging failure never blocks a session.
- **Manual fallback.** `llmlog.py archive --session-id <id> --transcript <path>` archives a session by hand (`CLAUDE.md`, `llm-logs/README.md`). `llmlog.py index` rebuilds every derived file.

Decisions: - One folder per session, instead of the single log of the `reproducible-llm-template` it was adapted from. - The JSONL transcript is kept as ground truth, and the derived CSV and Markdown files are regenerable. This is because the JSONL format is internal to Claude Code and may change.

Limits: - Only end-of-turn replies are reliably present in the transcript. - A session that dies without `SessionEnd` leaves its latest copy staged but uncommitted, until a later session commits it. - Redacting patient data or secrets from transcripts is left to the user.

Verified by TST010 - `tests/test_llm_logs.py`:-

`test_every_session_folder_is_indexed_and_vice versa` checks that every folder with a `transcript.jsonl` is listed in `sessions.csv` and holds `session.json`, `prompts.csv` and `PROMPTS-AND-RESPONSES.md`. Despite its name, it does not check that every indexed session has a folder, which TST010 also asks for. -

`test_index_regenerates_cleanly` runs `llmlog.py index` and requires exit 0, no `llmlog: error` and a `SESSIONS.md`.

Level	12.0	Parents	REQ012	Children	TST010
-------	------	---------	------------------------	----------	------------------------

13.0 Requirements traceability

DES013

Implements REQ013 - requirements shall be kept in Doorstop under `docs/traceability/` as a chain NEED -> REQ -> DES -> TST. Every normative REQ shall link to a need and have a design item, and every design item shall be covered by a test. `doorstop` validation shall pass before a release, and the published tree (Markdown per document and one PDF with the traceability matrix) shall be kept in the repository.

Design.

- *Documents.* Doorstop 3.2 (pinned in `requirements-dev.txt`) manages four documents, each with its own `.doorstop.yml`: `needs/` (NEED), `requirements/` (REQ, parent NEED), `design/` (DES, parent REQ) and `tests/` (TST, parent DES). All use `itemformat: yaml`, three digits and no separator. An item file carries:
 - `header`: the title
 - `text`
 - `level`
 - `links`: parent UID plus fingerprint
 - `normative`, `active`, `derived`, `ref`
 - `reviewed`: fingerprint
- *DES level.* The DES level was added later: one DES item per REQ, with the same number. TST items were relinked from REQ to DES, so a requirement is covered through its design item. `docs/traceability/README.md` gives the rules and commands.
- *Validation.* `.venv/bin/doorstop` checks links, levels and fingerprints and exits non-zero on errors. Doorstop itself only warns about missing text, missing child links, suspect links and unreviewed items, so the coverage rules are enforced by `pytest`. `setup.sh` runs the validation, and `CLAUDE.md` requires it before every commit.
- *Automated gate.* `.github/workflows/tests.yml` runs on every push and pull request (DES011): its `python` job runs `./setup.sh`, which fails on an invalid tree, and then `tests/test_traceability.py`. A change that breaks the chain or leaves the published Markdown stale turns CI red. There is no gate tied to a release: `.github/workflows/release-cadwellio.yml` runs neither Doorstop nor `pytest`, so a plugin release relies on CI having passed on the tagged commit.
- *Change control.* `doorstop review all` records reviews and `doorstop clear all` clears suspect links after a parent changes.
- *Published tree.*
 - `doorstop publish <DOC>` `docs/traceability/published/<DOC>.md` writes the committed Markdown copies `NEED.md`, `REQ.md`, `DES.md` and `TST.md`. The output is deterministic, so a fresh publish of an unchanged tree reproduces them byte for byte.
 - `doorstop publish all docs/traceability/published/html` writes the browsable HTML, which is gitignored.
 - `tools/doorstop_pdf.sh` validates the tree and runs `tools/doorstop_pdf.py`, which writes the committed `published/CwellEEGRead-traceability.pdf` (A4, CSS paged media). The script reads the tree with Doorstop's Python API and renders the item texts with Python-Markdown (a Doorstop dependency). It builds one HTML document: a cover with the commit hash, a table of contents with page numbers, one section per document with each item's level, UID, parents and children, and the full traceability matrix, one row per NEED -> REQ -> DES -> TST chain. `WeasyPrint (weasyprint==70.0` in `requirements-dev.txt`) renders it; it needs the Pango and HarfBuzz system libraries and the Liberation fonts, present on Debian/Ubuntu and in the Claude Code web container. The shell script sets `SOURCE_DATE_EPOCH` from the last commit touching `docs/traceability`, and `PYTHONHASHSEED=0`, so rebuilding an unchanged tree gives a byte-identical PDF.
 - *Why Doorstop.* Items are plain YAML files next to the code, reviewable in git diffs, and validated by a pinned command-line tool. `docs/research/` records no comparison with other tools.

Limits. Nothing checks that the committed PDF is current. It has to be regenerated by hand with `tools/doorstop_pdf.sh` after item changes, and CI does not rebuild it, because WeasyPrint's system libraries and fonts would make a byte comparison fragile across runners.

Verified by TST009 - `tests/test_traceability.py` runs: - `test_doorstop_validates`: Doorstop exits 0 with no errors. - `test_every_requirement_links_to_a_need` - `test_every_requirement_has_a_design` - `test_every_design_links_to_a_requirement_and_has_text` - `test_every_design_is_covered_by_a_test` - `test_every_requirement_is_covered_by_a_test_through_its_design` - `test_published_markdown_is_up_to_date`: a fresh `doorstop publish` of each document equals the committed `published/<DOC>.md`.

CI runs these on every push.

Level	13.0	Parents	REQ013	Children	TST009
-------	------	---------	--------	----------	--------

14.0 Anonymisation option

DES014

Implements REQ014 - a command-line option shall replace the patient identification fields with user values or placeholders and remove patient-identifying text from annotations, leaving the signal unchanged.

Design. Partly implemented; `README.md` lists anonymisation beyond the header as not yet handled.

- *Option.* `cwelleegread/__main__.py` offers `--anonymize` (a flag) and `--patient-name`, passed to `cwelleegread/edf.py: convert(anonymize=, patient=)`. The `patient` dict accepts `code` and `name`, but the CLI fills only `name`; there is no option for the code, birth date or sex.
- *Header.* Without the flag the patient code is the patient GUID from `MediaHeader.MediaDescriptor`; with it, `X`. The name is `--patient-name` or `X`; sex and birth date are always `X` (`cwelleegread/edfwrite.py: write_edf_plus()` defaults). The converter never reads the real name, birth date or sex - they sit in the encrypted `EEGRECORDINFO` blob (research note) - so these cannot leak from the recording with or without the flag. Still written with the flag: the first 8 characters of the record GUID in the recording field, and the start date and time.
- *Annotations.* With the flag, events of the types in `cwelleegread/edf.py: ANONYMIZED_EVENT_TYPES` (`Comment`, `UserEvent`, `Annotation`, `PatientEvent`: text typed by people) are written with their type name instead of their text. Other types (Persyst comments and so on) keep their text. No name matching is done; the converter has no copy of the name to search for.
- *Report.* `patient_guid` becomes null, and `events_skipped` shows those types by type name too (deleted comments included). The report still holds `record_guid` and the input path.
- *Signal.* The flag is consulted only in the header and annotation code; samples, physical ranges and records are the same as without it.
- *EEGLAB plugin.* No anonymisation option; `EEG.etc.cadwell` carries `patientGuid` and `recordGuid`.

Not yet designed: scrubbing all free-text types, shifting the start date, and CLI options for the patient code, sex and birth date.

Verified by TST007 - `tests/test_anonymisation.py` converts export 2 (a `Comment` and a `UserEvent` with typed text) through the CLI with and without `--anonymize --patient-name`: without the flag the patient GUID and the typed texts are in the EDF and report; with it they are nowhere in the EDF bytes or the JSON report, the patient field is `X X X <name>`, the two events are written by type name, the annotation count is unchanged and the digital samples of all 32 signals are identical.

Level	14.0	Parents	REQ014	Children	TST007
-------	------	---------	--------	----------	--------

15.0 Licence compatibility

DES015

Implements REQ015 - the repository licence shall be compatible with every third-party source ported into it. If BioSig (GPL-3.0-or-later) code is ported, the repository shall be relicensed under a GPL-compatible licence, and the decision and attribution shall be recorded in `docs/research/` and `LICENSE`.

Design. Nothing is ported, so the conditional part of the requirement has not been triggered. The design keeps it that way and makes the rule explicit.

- *Current licence.* The Unlicense (public domain), in the root `LICENSE`. `README.md` ("Licence") and `CLAUDE.md` ("Do not copy code from GPL sources (BioSig) into this repo without an explicit relicensing decision recorded in `docs/research/`") state it.
- *Decision record.* `docs/research/README.md` ("Licensing decision (REQ015)") keeps the Unlicense and commits to relicensing to GPL-3.0-or-later before any GPL decoder is ported. The decision would be recorded there.
- *Analysis behind it.* `docs/research/biosig-cadwell-reader.md` §5 weighs three options:

- A: port `sopen_cadwell_read.c` or `sopen_sqlite.c` and relicense. Rejected: the code has no working decoder; every path ends in `B4C_FORMAT_UNSUPPORTED`.
- B: clean-room reimplementation. Chosen. Format facts such as magic bytes and table names are cited, and no BioSig code, structure or names are copied.
- C: link libbiosig. Rejected: it would make the distributed whole GPL for no gain, since libbiosig cannot read Cadwell files.
- *Our own code.* The Python decoder and the EEGLAB plugin are written from the project's own analysis of the test recordings. The pure MATLAB/Octave SQLite reader `uses/EEGLAB/cadwellio/cadwell_sqlite_native.m` cites the SQLite file-format document. `docs/research/eeglab-fieldtrip-cadwell-reader.md` records which layout sources were consulted (MIT-licensed `sml-sqlite` and DC3 `sqlite-dissect`), and that the prototype `tools/sqlite_pure_reader.py` was written from the format description.
- *Plugin licence.* The plugin ships its own Unlicense copy, `uses/EEGLAB/cadwellio/LICENSE`, which `uses/EEGLAB/make_zip.sh` puts into the zip. The headers of the main `.m` files say "Public domain (Unlicense)". EEGLAB core is BSD-2 and allows plugins under other licences (research note).
- *Third-party components used but not ported.*
- Python runtime dependencies (`numpy`, `pyedflib`, `scipy`) are installed packages.
- The optional SQLite cross-check backends in `cadwell_sqlite.m` (`mksqlite`, the Database Toolbox or Octave `sqlite`, `py.sqlite3`) are called only if the user has them.
- The Apache-2.0 `sqlite-jdbc` jar is downloaded by `cadwell_get_jdbc.m` into `cadwellio/lib/`, which is gitignored (`lib/*.jar`). `make_zip.sh --with-jdbc` bundles it without adding a separate licence notice. That is a gap, but only in this optional mode.
- Morgoth (CC BY-NC 4.0) is cloned by `uses/Morgoth/install.sh` into a gitignored folder and is never redistributed. Its non-commercial term restricts use, not this repository's licence.
- *Future.* A FieldTrip contribution would be a GPL-3 copy of our own file, which the Unlicense permits. It is not done.
- *Register.* `docs/research/ported-files.yml` lists every ported third-party file with path, origin, SPDX licence and a line of its notice. It is empty (`ported: []`), and its header states the rule: only licences the Unlicense can carry (MIT, BSD-2/3-Clause, Apache-2.0, ISC, Zlib, CC0-1.0, Unlicense, public domain), with the notice kept.

Verified by TST012 - `tests/test_licence.py` checks that the root and plugin `LICENSE` files are the Unlicense; that every registered port exists, has an allowed licence, names its origin and contains its notice; and that no unregistered tracked source file (`.py`, `.m`, `.sh`, C and so on, outside `docs/`, `llm-logs/` and `testdata/`) contains third-party licence text (GPL/LGPL, Apache, MIT, BSD redistribution clause, MPL, SPDX identifier, "Copyright (c)"). With no ported file it passes because no source carries such text.

Level	15.0	Parents	REQ015	Children	TST012
-------	------	---------	------------------------	----------	------------------------

16.0 Downstream use scaffolds

DES016

Implements REQ016 - `uses/` shall hold one sub-folder per downstream use (Morgoth, SCORE-AI, EEGLAB), each with a README giving its status and prerequisites. The EEGLAB plugin can be packaged as a zip, shows a recording pause as a `Recording gap` event (padded) or a `boundary` event (concatenated, with later events moved earlier by the pause), and lists pauses in `EEG.etc.cadwell.gaps`.

Design. `uses/README.md` lists the three scaffolds with their status. Background: `docs/research/downstream-uses.md`.

Morgoth (scaffold, never run end to end). - `uses/Morgoth/install.sh` clones `bdsp-core/morgoth` (`MORGOTH_REF`, default `main`) into the gitignored `morgoth/`, and records the commit in `morgoth.commit`. It creates the conda environment `morgoth` (Python 3.12, PyTorch 2.4.1 on CPU or CUDA 12.4) and installs the upstream requirements. - `run.sh` runs event-level and EEG-level prediction per task. Options: `--cpu`, `--segment`, `--dry-run` (prints the commands) and `-h`. - `morgoth_to_json.py` merges Morgoth's CSV outputs into one JSON document. - The weights need credentialed BDSP access, and Morgoth is licensed CC BY-NC 4.0.

SCORE-AI (scaffold). No public SCORE-AI program exists. - `uses/SCOREAI/run_scoreai.py` fills the command template `SCOREAI_CMD ( {edf}, {json} )` and runs it. - `check_edf()` uses `pyedflib` to check the input: all 19 10-20 electrodes (with T7/T8/P7/P8 aliases); a warning for no ECG channel or less than 15 minutes; units in μV . - It writes JSON with the input's SHA-256, the check results, normalised probabilities, the raw output and provenance. Options: `--check-only`, `--dry-run`, `--batch`.

EEGLAB plugin cadwellio 0.3.0 (uses/EEGLAB/cadwellio/). Plain MATLAB/Octave. The main pieces:

- *Entry point*. `eegplugin_cadwellio.m` returns `vers = 'cadwellio0.3.0'` and adds *From Cadwell (.ezdataindex / converted EDF)* to the *Import data* menu; the import options dialog and the GNU Octave stand-ins are DES022.
- *Native SQLite reader*. `cadwell_sqlite_native.m` reads the whole file into memory and checks the header magic, page size and text encoding (Cadwell files use UTF-16LE). It walks each table's b-tree depth first with an explicit stack (interior pages type 5, leaf pages type 13), which yields rows in rowid order.
- It follows overflow chains using the file-format formulas (X, M, K).
- It decodes records by serial type, with a vectorised decoder for varint headers.
- It takes column names from the `CREATE TABLE` text and fills `INTEGER PRIMARY KEY` aliases.
- It keeps each value's storage class in `t.kinds`, and warns if a `-wal` file exists.
- No SQL, indexes or writes.
- *Backend switch*. `cadwell_sqlite.m` uses `native` by default. `mksqlite`, `sqlite` (Database Toolbox or Octave package), `jdbc` (xerial jar in `lib/`, fetched by `cadwell_get_jdbc.m`) and `python` (`py.sqlite3`, read-only) exist only to cross-check the native reader and for ad-hoc SQL.
- Why native: the scoping report (`docs/research/eeglab-fieldtrip-cadwell-reader.md`) rejected the library routes as defaults. The Database Toolbox is paid and does not return BLOBs [W]. MATLAB's Java support is being withdrawn [W]. MEX needs a binary per platform and MATLAB ABI.
- *Frame decoder*. `cadwell_decode_frame.m` is ported from the Python decoder in `cwellleegread/`.
- It checks the magic `0x033149BD`, finds the channel blocks by their 8-byte tag, and reads the channel count and start/end ticks (100 ns) from the header.
- `Samples = first + cumsum(int16 or int8 deltas) × scale`.
- It is vectorised: channels that share a delta type and length are decoded with one `typecast` and one `cumsum`.
- *Reader*.
- `cadwell_read_index.m` reads the GUIDs, track-0 channels, frames, gaps and clock correction. `cadwell_read_events.m` reads the events.
- `cadwell_read.m` finds the `.ezdataindex` and reads all `FrameInfo` blobs of each data file in one table read. It matches blobs to frames by `FrameKey` with `ismember`, because `containers.Map` is slow in Octave.
- It decodes the frames into a preallocated μV matrix. Labels come from the Apollo or Essentia headbox tables in `cadwell_layout.m`.
- *Pauses*.
- A frame number is one second, so a jump in frame numbers is a pause. `cadwell_read` records it in `gaps (startSample, seconds, startSec, padded)`.
- With `PadGaps` on, the pause is zero-filled; with it off, the segments are concatenated.
- `pop_cadwell.m: gap_events()` then writes, when padded, a `Recording gap` event at `startSample`; when concatenated, a `boundary` event at `startSample - 0.5`. Both have `duration = pause × srate` samples.
- Moving later events up: in `ticks` mode `event_sample()` maps onto the concatenated sample axis, and an instant inside a pause lands on the join. In `stamp` mode the pause lengths are subtracted.
- `EEG.etc.cadwell` is `rec` without data and events, so `gaps` is there in both modes.
- Deviation from REQ016: with `'impor tevent'`, `'off'`, only boundary events are written, so padded mode then has no `Recording gap` event.
- *Event timing (REQ021)*. The default, `ticks`, maps each event's `StartOffset` through the frames' tick spans (linear within a frame, nominal rate outside). `stamp` uses the wall-clock stamp relative to the first frame, as the vendor's EDF export does. Both onsets are kept.

- **Importer.** `pop_cadwell.m` builds the dataset from `eeg_emptyset` with single-precision data. Channel labels are electrode names with `.ref`, and `EEG.ref` is 'Cz'. It then runs `eeg_checkset` (eventconsistency, makeur) and writes a history string. It skips deleted events, bookkeeping event types and Photoc Stim. A `.edf` file is passed to `pop_biosig` or `pop_fileio`.
- **Self-test.** `cadwell_selftest.m` compares against reference data written by `tools/make_matlab_reference.py --tables:`
 - A: the decoder against Python, on stored frames.
 - B: a full read through every available backend.
 - D: the native reader's canonical dump of every table, byte for byte.
 - E: padded against concatenated reads.
 - F: tick onsets against stamp onsets plus the frame drift.
- C: export 1 against the vendor text export. The code allows 0.06 μ V, while TST013 says 0.05.
- **Zip.** `uses/EEGLAB/make_zip.sh` reads the version from `vers` and copies `*.m`, `README.md`, `LICENSE` and `octave/` into `dist/cadwellio<ver>/`, then zips it with the folder at the root. `--with-jdbc` also adds the jar. `dist/` is gitignored.
- **Release.** `.github/workflows/release-cadwellio.yml` runs on a `cadwellio-v*` tag, on a `release/cadwellio-v*` branch or by manual dispatch. It checks that the tag version equals `vers`, builds the zip and publishes it as the only asset with `softprops/action-gh-release@v2`. It does not run the self-test. The tag `cadwellio-v0.2.0` exists.

Not done yet: - The plugin has not been run in MATLAB (its menu and dialogs are exercised under Octave, TST017). - 0.2.0 is submitted to the EEGLAB plugin list (scn/eeGLAB issue 971, `SUBMISSION.md`); 0.3.0 is not released yet. - The FieldTrip reader exists only in research notes. - `uses/EEGLAB/README.md` no longer contains the words "status" or "prerequisite" (its section is headed "Requirements"), so the README check fails for EEGLAB.

Verified by TST013: - `tests/test_uses_scaffold.py` checks that each README mentions status and prerequisites. It does not run the scripts' `--help` or `--dry-run` modes, which TST013 also asks for. - `tests/test_octave_port.py` runs `cadwell_selftest` under GNU Octave on the three public exports. - `tests/test_eeglab_import.py` runs `pop_cadwell` on export 3 with EEGLAB functions (`EEGLAB_DIR`, `dipfit`). It checks one Recording gap or boundary event of 10 s, zeros in the padded pause, the shifted latencies, and a `pop_saveset / pop_loadset` round trip.

Level	16.0	Parents	REQ016	Children	TST013, TST018
-------	------	---------	--------	----------	----------------

17.0 Reproducible development environment

DES017

Implements REQ017 - all Python tooling shall be installed into a gitignored virtual environment from pinned requirement files by one `setup.sh`, so a fresh checkout works after one command.

Design.

- `setup.sh`. A bash script with `set -euo pipefail` that runs from its own folder. It:
 - creates `.venv` with `$PYTHON` (default `python3`), but only if `.venv/bin/python` is missing, so re-runs are cheap and idempotent;
 - upgrades pip and installs `requirements-dev.txt`;
 - marks `.claude/hooks/*.sh` and `llm-logs/tools/llmlog.py` executable;
 - prints the Doorstop version, runs `.venv/bin/doorstop` (an invalid requirement tree fails the setup) and `llmlog.py index`, which regenerates the derived `llm-logs` files as a side effect.
- **Requirement files.**
 - `requirements.txt` (runtime): `numpy>=1.26`, `pyedflib>=0.1.38`, `scipy>=1.11`.
 - `requirements-dev.txt` (tooling): includes `-r requirements.txt` and adds `doorstop==3.2`, `pytest>=8`, `pyyaml>=6`, `weasyprint==70.0` (the Doorstop PDF) and `mne>=1.6` (the second EDF reader in the tests).
- `pyproject.toml`: package metadata with the same runtime dependencies and the `cwellleegread` console script; `setup.sh` installs it editable after the requirements.

- *Version pinning.* Only Doorstop is pinned exactly; the other packages have minimum versions only. There is no lock file, no hashes and no container image, so two fresh setups can resolve different versions.
- *Ignored and test configuration.* `.gitignore` excludes `.venv/`, `__pycache__`, `.pytest_cache/` and `.cache/`. `pytest.ini` sets `testpaths = tests`, `pythonpath = .` and the `testdata` marker. Tests that need the private recordings skip through `tests/conftest.py`.
- *Cloud sessions.* `.claude/hooks/session-start.sh` is a `SessionStart` hook in `.claude/settings.json` with a 600 s timeout. It acts only when `CLAUDE_CODE_REMOTE=true`. It:
 - runs `./setup.sh`;
 - `apt-get` installs `octave`, `zip` and `unzip` if they are missing;
 - makes a sparse, depth-1 clone of `scn/eeglab (functions/)` into `.cache/eeglab-src` and a clone of `scn/dipfit` into `.cache/dipfit`;
 - exports `EEGLAB_DIR` and `DIPFIT_DIR` through `CLAUDE_ENV_FILE`.

This makes the Octave and EEGLAB tests runnable there. None of these non-Python tools is version-pinned. Local sessions are left alone, and those tests skip when Octave or EEGLAB is absent. - *Out of scope.* The Morgoth scaffold has its own conda environment (uses `Morgoth/install.sh`). `setup.sh` does not handle it.

Verified by TST011 - the python job of `.github/workflows/tests.yml` runs `./setup.sh` on a fresh checkout (Python 3.11 and 3.12), checks `doorstop --version`, `git check-ignore .venv` and `cwelleegread --version`, and runs the test suite in that `.venv`. `tests/test_environment.py` checks in any checkout that `.venv`, `.cache`, `testdata/private/` and the egg-info are ignored, that the installed Doorstop equals the pin, that `setup.sh` passes `bash -n`, is executable in git and installs the package, and that `pyproject.toml` and `requirements.txt` list the same runtime dependencies.

Level	17.0	Parents	REQ017	Children	TST011
-------	------	---------	------------------------	----------	------------------------

18.0 Clear failure on unsupported input

DES018

Implements REQ018 - input that cannot be fully converted shall stop the program with a diagnostic naming the problem and a non-zero exit code, without leaving a partial EDF that could pass for complete.

Design.

- *No partial output.* `cwelleegread/__main__.py: cmd_convert()` refuses an existing output without `--force` (exit 1), opens the recording, and has `cwelleegread/edf.py: convert()` write to `<output>.part`; only after `convert()` returns is the file renamed with `os.replace` (atomic on one file system). On any `Exception` the `.part` file is removed and the exception re-raised. Inside `convert()` all decoding and checks run before `cwelleegread/edfwrite.py: write_edf_plus()` opens the file, and the writer validates every header field before writing. A `.part` file can remain only when the process is killed or interrupted (`KeyboardInterrupt` is not an `Exception`); its name marks it incomplete. Files are opened read-only (`mode=ro&immutable=1`), so the input is never changed.
- *Diagnostics.* `main()` turns `FileNotFoundError`, `KeyError`, `ValueError` and `sqlite3.DatabaseError` into `error: <message>` on `stderr` and exit 1; any other exception prints its traceback and also returns 1. In batch mode each failure is reported the same way and the batch continues (DES006). Conditions detected with a message: a storage schema version outside `SUPPORTED_SCHEMA_VERSIONS` (`UnsupportedVersionError` from `CadwellRecording.check_supported()`, called first in `convert()`); `--allow-unsupported` downgrades it to a report warning; DES001); a corrupt or truncated SQLite file (`error: unreadable Cadwell database: ...`); no `.ezdataindex`, or several records in one folder (`cwelleegread/ezdata.py: open_recording()`); a missing `.ezdata` file or frame key (`CadwellRecording.frames()`); wrong frame magic (`FRAME_MAGIC 0x033149BD`), no channel blocks, channel-count mismatch, unknown delta type (not 1 or 2) (`decode_frame()`); a recording shorter than one second; an over-long header field or a start outside 1985-2084 (`edfwrite.py`).
- *Limits.* The version check is on the storage schema; a new Cadwell software version with an unchanged schema but a different frame encoding would still fail only in `decode_frame()`. An unknown amplifier type is not an error; it falls back to a label table with a report warning (DES004).

- *EEGLAB plugin.* `cadwell_read.m`, `cadwell_read_index.m` and `cadwell_decode_frame.m` raise MATLAB errors with identifiers (`cadwell_read:notFound`, `cadwell_read:missingData`, `cadwell_read:missingFrame`, `cadwell_decode_frame:magic`, `cadwell_decode_frame:deltaType` and others); the plugin writes no file.

Verified by TST006 - `tests/test_convert_public.py: test_cli_behaviour` runs the CLI on a valid export (exit 0, JSON report), on a non-existent path (non-zero exit, `error` on stderr, no output file) and on an existing output without `--force` (non-zero exit); `tests/test_cli.py` runs it on a copy of export 1 whose schema is set to 9.9 (refused, no traceback, no `.part`; converted with `--allow-unsupported`) and on a truncated index (one-line diagnostic, no output), and runs a batch with one unsupported recording.

Level	18.0	Parents	REQ018	Children	TST006
-------	------	---------	--------	----------	--------

19.0 Recording gaps and discontinuities

DES019

Implements REQ019 - recording gaps shall be detected from the frame numbering and, by default as the vendor does, written into a continuous EDF+C as digital zero at frame boundaries; as an option outside vendor mode, as a discontinuous EDF+D without the gap seconds; with a gap annotation, and listed in the report.

Design.

- *Detection.* A track-0 frame is one second of the amplifier clock (frame n covers ticks n to $n+1$ s). `cwelleegread/edf.py: read_padded()` walks the stored frames in `Offset` order; when a frame number exceeds the previous one by more than 1, the missing seconds become `missing x rate zero` samples per channel at that frame boundary, `per_frame` gets nominal-rate entries, and `(first padded sample, seconds)` is appended to the gap list. The `GapInfo` rows (`cwelleegread/ezdata.py, CadwellRecording.gaps`) are read and copied to the report but not used for detection; on export 3 both give frames 328-337 (10 s).
- *Why not the events.* The vendor's `Stop Recording/Start Recording` events do not mark the data edges: data end 0.5-2 s before `Stop` and resume up to 1 s after `Start` (research note, "Where the vendor's `Stop/Start Recording` events sit"). They are exported as ordinary annotations and fall inside the zero run, as in the vendor EDF.
- *Policy.* `convert(gaps=...)`, CLI `--gaps pad|discontinuous`, default `pad`. EDF+D is opt-in because EDFlib/pyedflib refuses a discontinuous file, and MNE-Python (1.13) reads it as contiguous without a warning, which misplaces all data after a gap relative to its annotations; zero padding is read correctly by every reader and matches the vendor. `discontinuous` is refused in vendor mode (it reproduces the vendor's EDF+C) and with `--format edf`; on a recording without gaps it gives EDF+C. The leading padding of `--start-at record-origin` is not a gap and stays in either case. The report states `gaps_policy`, `records_written` and `records_omitted_in_gaps`.
- *Discontinuous (EDF+D).* The padded stream is cut into 1-s records as before; every record lying wholly inside a gap's padding is left out, and `cwelleegread/edfwrite.py: write_edf(edf_type='edf+d')` writes the rest with their true onsets (whole seconds after the data start plus the sub-second start) in each record's time-keeping TAL. On Essentia (500 samples per frame) the gap is record-aligned and exactly its seconds disappear (export 3: 1207 of 1217 records). If a gap does not start on a record boundary (Apollo's 248-251-sample frames), the records at its edges keep their few padded zeros; the count is a report warning. Raw mode adds `Recording gap N s` with duration N at the gap start; events inside the gap (the vendor's `Stop/Start Recording`) keep their onsets and are stored in the record before the gap. Readers: EDFbrowser handles EDF+D; EDFlib/pyedflib refuses it and MNE-Python (1.13) reads it as contiguous without a warning, which puts everything after a gap early by the gap length (DES002). Hence the opt-in.
- *Padded (EDF+C).* `write_edf()` writes the padded stream as EDF+C; zero maps to digital 0 under the symmetric physical ranges. In raw mode `convert()` adds `Recording gap N s` (padded with zeros) with duration N at the gap start; vendor mode pads but adds no annotation, since the vendor EDF has none. `--start-at record-origin` pads leading missing frames the same way. Tick-based onsets (`event_sample()`) run through the pad at the nominal rate, keeping events on wall-clock time. `vendor_highpass()` filters each segment separately, primes the filter again after a gap and restores the zeros.

- **Report.** `gaps` (the `GapInfo` rows), `gaps_padded` (start second, length), `frames_padded`, the policy fields above and a warning with count and total seconds (padded, or left out of the EDF+D file).
- **Alternatives and limits.** Concatenation is offered only by the EEGLAB plugin: `cadwell_read.m` `'PadGaps'` false (the `pop_cadwell` default) joins the segments with `boundary` events, true pads with zeros and `Recording gap` events; both list pauses in `EEG.etc.cadwell.gaps`. Data the vendor discarded around a pause cannot be recovered. Gaps are assumed to be whole missing frames at the nominal rate.
- **Segment-wise comparison.** The vendor text export omits the gap, so equivalence against it is done by frame number (`tests/test_export3_gap.py: test_full_text_export_equivalence_at_500hz`, `tests/test_vendor_exports_filtering.py`).

Verified by TST015 - `tests/test_export3_gap.py` (with `gaps='pad'`): `test_gap_in_index` checks the `GapInfo` row and the missing frames 328-337, and `test_gap_is_padded_and_annotated` checks 1217 records, 10 padded frames, zeros at 327-337 s with data on both sides, the gap annotation, the Stop/Start Recording annotations and Stop offset, and the report's `gaps_padded`. `tests/test_edf_formats.py:`

`test_gaps_padded_by_default_and_edf_plus_d_on_request` checks that the default is EDF+C and that `--gaps discontinuous` gives EDF+D, 1207 records whose onsets skip seconds 327-336, samples equal to the padded file's without those records, the `Recording gap 10 s` annotation and the Stop/Start Recording events;

`test_edf_plus_d_rules` checks the writer's onset checks and the refusal in vendor mode;

`test_discontinuous_without_gaps_stays_continuous` checks a gapless recording.

Level	19.0	Parents	REQ019	Children	TST015
-------	------	---------	--------	----------	--------

20.0 Sample clock and resampling policy

DES020

Implements REQ020 - keep every raw sample by default (nominal rate, only the trailing partial second dropped, effective rate and drift reported) and offer a vendor-compatible mode reproducing the vendor's EDF export sample by sample, recording the policy in the recording-additional field and the report.

Design. `cwelleegread/edf.py: convert(mode=...)`, CLI `--mode raw|vendor` (default raw). Both modes start from `read_padded()`: all stored frames, gaps padded with digital zero at the nominal rate. Raw mode only truncates to whole seconds and declares the nominal rate, so on Apollo (248/250/251 samples per frame) the EDF duration exceeds the stamped duration by the drift; accepted because every sample stays and events follow the sample clock (DES021). The report gives `samples_per_frame`, `raw_samples`, `written_samples`, `samples_dropped_at_end` and `effective_rate_hz` (samples of all frames but the last over the first-to-last frame-stamp span); drift appears only through that rate. Vendor mode runs `vendor_highpass()` then `vendor_resample()`. `vendor_resample()` keeps all frames but the last ($N_{out} = \text{frames} \times \text{rate}$) and removes the surplus $S = N_{in} - N_{out}$ at period $T = \text{ceil}(N_{in} / (S + 1))$ in output coordinates, replacing the two output samples at each removal by two-point means (export 1: $T = 1224$); constant-ratio interpolation was tested and does not match. `vendor_highpass()` applies `scipy.signal.butter(2, VENDOR_HIGHPASS_HZ = 0.16, 'high')` causally (`lfilter`) to each contiguous segment, primed on the time-reversed first min(20 s, segment) including the first sample, re-primed after each gap, gap seconds reset to zero; zero-state, steady-state and constant-extension start-ups were rejected (hundreds to thousands of steps off). `--highpass auto` filters only vendor-mode Essentia recordings (the Apollo vendor EDF was unfiltered; headbox or software version as the cause is unknown), `on|off` overrides. `--start-at record-origin` pads the leading missing frames. Physical range: `layout.py: HEADBOXES` gives $\pm 562500 \mu\text{V}$ (Apollo) or $\pm 32767 \times \text{UNIT_UV} = \pm 23919.27 \mu\text{V}$ (Essentia; the vendor writes -23919.0/23919.03). Events follow `SKIPPED_EVENT_TYPES/TEXTS` and stamp timing. The policy goes into recording-additional as `CwellEEGRead_<mode>_<timezone>` (34 characters) and into the report (`mode`, `highpass_hz`, `start_at`, `event_timing`).

Deviations found. With `--start-at record-origin`, `effective_rate_hz` counts the leading padded samples but not their time (export 2: 500.57 instead of 500.05 Hz). `restart_filter_at_gaps` is accepted but unused. Recording-additional holds the mode only. A vendor export starting at a user-chosen frame cannot be reproduced.

Verified by TST003 - vendor-mode sample equivalence within one quantisation step: export 1 (11000 samples and the removed indices, `tests/test_convert_public.py`) and exports 2 and 3-withfilter (480000 and 608500 samples with padded gap and record-origin start, `tests/test_vendor_exports_filtering.py`); raw fidelity via TST005.

Level	20.0	Parents	REQ020	Children	TST003
-------	------	---------	--------	----------	--------

21.0 Event placement on the sample clock

DES021

Implements REQ021 - events shall by default be placed on the amplifier sample clock from their tick offsets, with wall-clock-stamp placement available as `--event-timing stamp` and default in vendor mode, the choice stated in the report, and the same option and default in the EEGLAB plugin.

Design. `cwelleegread/edf.py: read_padded()` fills a frame table with one `(start_ticks, end_ticks, first_sample, n_samples)` per stored frame (blob ticks, 100 ns from the record origin; `first_sample` on the padded axis). `event_sample(ticks, frame_table, rate)` finds the frame by bisection on start ticks and returns a fractional sample: inside a frame, linear between its start and end ticks over its `n` samples (absorbing the 248-251-sample, jittered Apollo frames); in a padded gap, after the last frame or before the first, at the nominal rate from the nearest frame edge. In `convert()`, `event_timing='auto'` (CLI `--event-timing auto|ticks|stamp`) resolves to `ticks` in raw mode and `stamp` in vendor mode. Ticks: `onset = event_sample(StartOffset) / rate` plus leading padded seconds, `duration = (EndOffset - StartOffset) / 1e7 s`. Stamp: `onset = StartTime` minus the first stored frame's `TimeStamp`, `duration = EndTime - StartTime`. The EDF start time stays stamp-based (first frame stamp plus the `PcTimeSync` correction) either way, and the report field `event_timing` states the placement. Rationale (research note, "Two clocks"): samples are on the tick axis by construction; on Essentia the stamp clock falls about 96 ppm behind (0.35 s per hour), so stamp placement is early, as the flash response latency confirms (110-130 ms on ticks, 210-270 ms on stamps). Stamp stays the vendor-mode default so vendor equivalence (TST003, TST004) holds. The EEGLAB plugin: `uses/EEGLAB/cadwellio/cadwell_read.m` takes `'EventTiming'` (`ticks` default, or `stamp`) with a local `event_sample()` mirroring the Python rule, which also puts an instant inside a concatenated pause (`'PadGaps'` off, the plugin default) on the join; stamp placement with concatenated pauses shifts later events by the pause length. `pop_cadwell.m` exposes it as `'eventtiming'` (default `ticks`).

Vendor mode with ticks. `vendor_resample()` removes samples, so a raw sample position is mapped onto the resampled axis: `vendor_raw_positions(removed, n_out)` gives the raw position of every output sample (copied samples on their raw index, the two two-point means of a removal at raw `q` at `q-1.5` and `q-0.5`), and `raw_to_output_position()` interpolates linearly through it (nominal rate before the start and past the end). Without this the onsets were late by the samples removed before them (export 1: 8 samples, 32 ms, at the end). Stamp placement, the vendor-mode default, is unchanged.

Limits. The plugin has no vendor mode, so its `ticks` default matches the converter's raw default.

Verified by TST016 - `tests/test_event_timing.py`:

`test_vendor_mode_ticks_follow_the_removed_samples` checks on export 1 that the position map reproduces the resampled data exactly and that every vendor-mode tick onset lies within one sample of its raw position minus the removals before it (the last event moves by more than 20 ms); `test_ticks_versus_stamp` converts exports 2 and 3 in raw mode with every event, checks the report field, the auto defaults, that ticks minus stamp onsets equal the frame drift within 1 ms, and that the flash response peaks before 150 ms with ticks and after 170 ms with stamps; `cadwell_selftest.m` check F (run by `tests/test_octave_port.py`) checks the plugin's tick-stamp difference against the frame drift.

Level	21.0	Parents	REQ021	Children	TST016
-------	------	---------	--------	----------	--------

22.0 EEGLAB plugin import dialog

DES022

Implements REQ022 - from EEGLAB's menu, a dialog for the import options (events, recording pauses, event timing; events only for EDF) after the file dialog, defaults preselected, options in the dataset and the history, EEGLAB's naming dialog afterwards, Cancel creates nothing, no dialog when a path is given; GNU Octave stand-ins for what EEGLAB's interface needs.

Design. `uses/EEGLAB/cadwellio/pop_cadwell.m`: without a path it calls `uigetfile`, then, when no options were passed, the local `options_dialog(filename)`. That builds an EEGLAB `inputgui` titled *Import Cadwell EEG -- pop_cadwell()* with a checkbox tagged `importevent` (on) and, unless the file is an `.edf`, two popup menus tagged `padgaps` (*Join the segments, mark each pause with a boundary event* first, as `'padgaps'`, `'off'`) and `eventtiming` (*Amplifier sample clock (recommended)* first, as `'ticks'`), and a Help button (`pophelp('pop_cadwell')`). The

values are read from `inputgui`'s tag structure and turned into the key/value pairs of the command line, so the rest of `pop_cadwell` and its history string are shared with scripted use. Cancel (empty result) returns `EEG = []`, `com = ''`, which EEGLAB's `eeglab_new` treats as no new dataset. The plugin's menu callback is unchanged (`[EEG LASTCOM] = pop_cadwell;` followed by EEGLAB's `catchstrs.new_and_hist`), so EEGLAB itself runs `pop_newset` and its naming dialog. Version `cadwellio0.3.0`.

GNU Octave. Two faults, found by driving EEGLAB's interface under Octave 8.4 (EEGLAB of September 2026), neither in the plugin: - `eeglab>eeg_mainfig` reads a variable `vers` that it only sets when `computer()` starts with GLN, MAC or PCW (added to `eeglab.m` in August 2025); Octave's `computer()` never does, so the main window does not open. `cadwellio/octave/vers.m`, a function returning `version()`, is found in its place, since Octave resolves an unset name to a function at run time. It must be on the path before EEGLAB starts (the plugin loads after the main window), so the READMEs tell Octave users to add the folder in `~/.octaverc`. The fix belongs in EEGLAB. - `eeglab_new`, run after every import from the menu, calls `contains`, which Octave lacks. `cadwellio/octave/contains.m` implements it for character vectors and cell arrays (patterns as text or cell array, `'IgnoreCase'`); `eegplugin_cadwellio` adds `octave/` to the path under Octave only when no `contains` exists. - EEGLAB adds only the plugin's own folder to the path, not sub-folders, so neither stand-in is used under MATLAB. `make_zip.sh` ships `octave/`. Debian/Ubuntu Octave also needs `fonts-freefont-otf` to draw text (documented, installed in CI and by the SessionStart hook).

Verified by TST017 - `tests/test_eeglab_gui.py`; TST018 - screenshots of the dialogs and of the imported EEG (`tools/eeglab_screenshots.py`, `tests/test_eeglab_screenshots.py`), checked by hand against `docs/screenshots/eeglab/README.md`.

Level	22.0	Parents	REQ022	Children	TST017 , TST018
-------	------	---------	------------------------	----------	---

Test specifications (TST)

How each design is verified. Each test specification links to the design items it proves and names the automated test that implements it.

1.0 EDF structural validity

TST001

Convert each test recording and open the result with two independent EDF readers (pyedflib and MNE-Python). Pass if the file is read without header errors, reports the expected number of channels, records and duration, and the samples read back equal the decoded samples within the declared resolution, with both readers. EDF+D files, which EDFlib/pyedflib refuses and MNE-Python reads as if contiguous, are checked with the test suite's own record-level reader for the record onsets. Implemented for the public exports in tests/test_convert_public.py (raw and vendor modes, pyedflib) and tests/test_edf_formats.py (MNE-Python read-back, plain EDF, EDF+D, the prefilter field).

Level	1.0	Parents	DES002 , DES010	Children	-
-------	-----	---------	---	----------	---

2.0 Header equivalence with native EDF export

TST002

For each test recording compare our EDF header with the native Cadwell EDF export: channel labels (after the documented mapping), sampling rates, physical dimensions, physical/digital ranges, start date/time, patient and recording identification. Pass if all match exactly, except fields that the mapping table documents as intentionally different.

Level	2.0	Parents	DES003 , DES004 , DES008	Children	-
-------	-----	---------	--	----------	---

3.0 Sample equivalence with native EDF export

TST003

For each test recording with an unfiltered vendor text export, prove raw fidelity against the text export (TST005). For each recording with a vendor EDF export, convert in vendor-compatible mode with the same start (first frame, record origin or user time) and compare every sample of every channel with the native EDF: the sample counts must match and the maximum absolute difference must not exceed one digital quantisation step of the native export (a small allowance for the vendor's asymmetric physical range is documented). Implemented: cadwell-export1 (Apollo, tests/test_convert_public.py, 11000/11000 samples) and cadwell-export2 and 3-withfilter (Essentia, tests/test_vendor_exports_filtering.py, 480000 and 608500 samples including a padded gap). Report the maximum and mean difference per channel.

Level	3.0	Parents	DES003 , DES008 , DES020	Children	-
-------	-----	---------	--	----------	---

4.0 Annotation equivalence with native EDF export

TST004

For each test recording compare the EDF+ annotation lists (onset, duration, text) of our export and the native export. Pass if, in vendor-compatible mode, the lists are equal after rounding onsets to the millisecond (the vendor's event policy - deleted events, amplifier bookkeeping types and the individual photic flashes omitted - is reproduced), and, in raw mode, the native list is a subset of ours; list every unmatched annotation on failure. Implemented for cadwell-export1, 2 and 3-withfilter.

Level	4.0	Parents	DES005 , DES008	Children	-
-------	-----	---------	---	----------	---

5.0 Sample equivalence with native CSV/text export

TST005

For each test recording parse the native CSV/text export, map its column order (amplifier input order) to EDF channel labels, and compare the physical values with our decoded samples over the full common range, sample by sample

without any alignment (the text export keeps every raw sample and omits recording gaps, so compare frame by frame). Pass if the maximum absolute difference is within the numeric precision of the text export (0.05 μ V for 4 decimals of mV) plus a documented allowance for the microvolt scale constant, and every column maps to exactly one channel. Implemented for the public exports: cadwell-export1 (7755 rows), cadwell-export2 (15500 rows) and the whole of cadwell-export3 (603000 rows, across the break) in tests/test_ezdata_public.py, tests/test_export3_gap.py and tests/test_vendor_exports_filtering.py.

Level	5.0	Parents	DES009	Children	-
-------	-----	---------	--------	----------	---

6.0 Command-line behaviour

TST006

Run the CLI on a valid recording, on a non-existent path, on a corrupt file and on an unsupported version, and run the batch mode on a folder holding several recordings, one of them unsupported. Pass if the valid run returns exit code 0 and a valid JSON report when requested; every invalid run returns a non-zero exit code, prints a one-line diagnostic (no traceback) and leaves no output file behind; an unsupported version converts only with `--allow-unsupported`, with a report warning; the batch converts every other recording, names the outputs after the records, writes a summary listing the failure and exits non-zero; and the installed `cwelleegread` command runs. Implemented in tests/test_convert_public.py (`test_cli_behaviour`) and tests/test_cli.py.

Level	6.0	Parents	DES006, DES018	Children	-
-------	-----	---------	----------------	----------	---

7.0 Anonymisation

TST007

Convert a test recording with the anonymisation option. Pass if the patient identification fields contain only the supplied or placeholder values, no annotation contains the original patient name or identifier, and the signal data is identical to a conversion without the option. Implemented in tests/test_anonymisation.py on cadwell-export2 (a Comment and a UserEvent with typed text): the patient GUID and the typed texts appear in the file and report without the option and nowhere in the EDF bytes or the JSON report with it, the patient field is `X X X <name>`, and the digital samples of all 32 signals are identical.

Level	7.0	Parents	DES014	Children	-
-------	-----	---------	--------	----------	---

8.0 Test data manifest and graceful skip

TST008

Verify that every test recording listed in the manifest matches its recorded sizes and SHA-256 checksums - the committed public recordings on every run, the private ones whenever `testdata/private/` is present - and that when the private folder is absent all tests needing it are reported as skipped with an explanatory message rather than failing. Implemented in tests/test_manifest.py (public entries) and the `testdata` fixture in tests/conftest.py (private entries).

Level	8.0	Parents	DES007, DES011	Children	-
-------	-----	---------	----------------	----------	---

9.0 Traceability validation

TST009

Run `doorstop` on `docs/traceability/`. Pass if it reports no errors, every normative REQ item links to at least one NEED item, every normative REQ item has a DES item with non-empty design text linking to it, every DES item is linked from at least one TST item, so every requirement is covered by a test through its design, and the committed `published/*.md` equal a fresh `doorstop publish`. Implemented in tests/test_traceability.py.

Level	9.0	Parents	DES013	Children	-
-------	-----	---------	--------	----------	---

10.0 LLM log integrity

TST010

Verify that every session folder under `llm-logs/` contains a transcript and is listed in `llm-logs/sessions.csv`, that every session in `sessions.csv` has a folder, and that `llm-logs/tools/llmlog.py` index regenerates the derived files without error.

Level	10.0	Parents	DES012	Children	-
-------	------	---------	--------	----------	---

11.0 Environment setup

TST011

On a fresh clone run `./setup.sh`. Pass if it completes without error, `.venv/bin/doorstop --version` prints the pinned version, `.venv` is ignored by git, the `cwellleegread` command is installed and the test suite passes in that environment. Automated as the `python` job of `.github/workflows/tests.yml` (fresh checkout on every push, Python 3.11 and 3.12); the checks that need no fresh clone (ignore rules, Doorstop pin, `setup.sh` syntax and mode, runtime dependencies of `pyproject.toml` equal to `requirements.txt`) are in `tests/test_environment.py`.

Level	11.0	Parents	DES017	Children	-
-------	------	---------	--------	----------	---

12.0 Licence compatibility check

TST012

Inspect the `LICENSE` file and the headers of every ported third-party source file. Pass if the repository licence permits every ported file's licence and each ported file carries its original copyright and licence notice. Implemented in `tests/test_licence.py`: the root and plugin `LICENSE` files are the Unlicense; every file registered in `docs/research/ported-files.yml` exists, has a licence the Unlicense can carry and contains its notice; and no unregistered tracked source file contains third-party licence text (GPL, LGPL, Apache, MIT, BSD, MPL, SPDX or copyright notices).

Level	12.0	Parents	DES015	Children	-
-------	------	---------	--------	----------	---

13.0 Downstream use scaffolds present

TST013

Check that `uses/Morgoth`, `uses/SCOREAI` and `uses/EEGLAB` exist, each with a `README` that states status and prerequisites, and that each provided script runs its `--help` or dry-run mode without error. For the `EEGLAB` plugin additionally run its self-test under GNU Octave when available (`tests/test_octave_port.py`): the `MATLAB/Octave` frame decoder must equal the Python decoder bit for bit on stored frames of every public export, the read through every available `SQLite` backend (always the pure `MATLAB/Octave` reader `cadwell_sqlite_native.m`) must yield the same index, labels, events, gaps and samples as the Python reader, the native `SQLite` reader's canonical dump of every table of every file must equal the dump written by Python's `sqlite3` byte for byte, for a recording with a pause the padded and concatenated reads must agree (zeros at the pause, pause position and length, event onsets after the pause shifted by its length), and export 1 must equal the vendor's text export within $0.05 \mu\text{V}$. The importer's dataset must pass `EEGLAB`'s `eeg_checkset` and a `pop_saveset / pop_loadset` round trip with `EEGLAB`'s functions on the Octave path; with `'padgaps', 'off'` exactly one `boundary` event per pause must be present, with duration equal to the removed samples.

Level	13.0	Parents	DES016	Children	-
-------	------	---------	--------	----------	---

14.0 Supported-version coverage

TST014

Check that the documented list of supported Cadwell storage schema versions (`cwellleegread/ezdata.py`: `SUPPORTED_SCHEMA_VERSIONS`) is non-empty, that every listed version is represented by at least one recording in the test data manifest, and that each public manifest recording's data carries the schema version the manifest states. Implemented in `tests/test_manifest.py`; the refusal of other versions is tested under TST006.

Level	14.0	Parents	DES001	Children	-
-------	------	---------	--------	----------	---

15.0 Gap handling

TST015

Using a test recording that contains at least one acquisition gap (cadwell-export3: 10 s), convert in raw mode with the default gap handling. Pass if the file is EDF+C, the number of EDF records equals the frame-number span, the padded seconds read back as digital zero at exactly the missing frame numbers, the samples on both sides of the gap are unchanged, the gap annotation and the vendor's Stop/Start Recording events appear at the correct offsets, and the conversion report lists the gap. Convert it again with `--gaps discontinuous`. Pass if the file is EDF+D, the record onsets skip exactly the gap seconds, the samples equal the padded file's without the gap records, and the gap and Stop/Start Recording annotations are present. Implemented in tests/test_export3_gap.py and tests/test_edf_formats.py.

Level	15.0	Parents	DES008, DES019	Children	-
-------	------	---------	----------------	----------	---

16.0 Event timing on the sample clock

TST016

Using a recording with photic stimulation (cadwell-export2 and 3: 244 Photic Stim flash events each), convert in raw mode with every event exported, once with `--event-timing ticks` and once with `stamp`. Pass if: the difference between the two onsets of every flash equals the drift between the frame time stamps and the frame ticks at that point of the recording within 1 ms (78 ms at 13.5 min in export 2, 96 ms at 17 min in export 3); the flash-locked average of O1 and O2 (referenced to Cz) peaks before 150 ms after the flash with `ticks` (a flash VEP) and later than 170 ms with `stamp` (the stamp-placed flash is early, so the response appears late); the report states the placement; and vendor-mode annotations are unchanged (the existing equivalence tests against the vendor EDF, TST003/TST005, keep passing). In vendor mode with `ticks` (cadwell-export1), pass if the map from raw to resampled sample positions reproduces the resampled data exactly and every onset lies within one sample of its raw tick position minus the samples removed before it. For the EEGLAB plugin the self-test checks that `EventTiming ticks` and `stamp` differ by the drift the frames show at each event (tolerance 5 ms, widened by the stamp jitter on Apollo recordings). Implemented in tests/test_event_timing.py and cadwell_selftest check F.

Level	16.0	Parents	DES005, DES021	Children	-
-------	------	---------	----------------	----------	---

17.0 Import through the EEGLAB menu

TST017

Start EEGLAB's graphical interface under GNU Octave on a virtual display (xvfb) from a private EEGLAB tree with the plugin copied into `plugins/`, with `octave/vers.m` on the path beforehand as documented. Replace only the system file dialog (`uigetfile`, returning public export 3) and wrap EEGLAB's `inputgui` so that each dialog is drawn by the real `inputgui` ('plot' mode), answered by widget tag and read back with 'getresult'. Pass if: the plugin is loaded, *From Cadwell (.ezdataindex / converted EDF)* is present under *Using EEGLAB functions and plugins* and enabled before any dataset exists; under an Octave without `contains` the plugin has put its stand-in on the path; choosing the item shows the options dialog, with two choices in each popup menu, and then EEGLAB's naming dialog; with *Fill each pause with zeros* and *Wall-clock stamps* chosen and a name typed, exactly one dataset is stored under that name, with one Recording gap event of 10 s and no boundary event, stamp event timing, and a history containing the `pop_cadwell` call with 'padgaps', 'on' and 'eventtiming', 'stamp'; no EEGLAB error is raised; and choosing the item again and pressing Cancel in the options dialog stores no further dataset. Skipped when Octave, xvfb, EEGLAB or the export is missing, or Octave cannot draw text on the virtual display.

Level	17.0	Parents	DES022	Children	-
-------	------	---------	--------	----------	---

18.0 EEGLAB plugin screenshots, manually verified

TST018

Screenshots with manual verification. Automated part: run `tools/eeglab_screenshots.py`, which starts EEGLAB's graphical interface under GNU Octave on a virtual display (Xvfb) with the plugin installed in `plugins/` as from its zip. It chooses *From Cadwell* in the menu and works the real dialogs from outside with `xdotool`: it types public export 3's `CadLink/Data` folder and `.ezdataindex` into the file dialog, and presses *Ok* in the options dialog and in EEGLAB's naming dialog, keeping the defaults. It then shows the dataset with `pop_eegplot`, first page, 10 s window.

Pass if four screenshots are written (the file dialog, the options dialog, the naming dialog and the EEG), each at least the expected size and not blank (`tests/test_eeglab_screenshots.py` ; CI keeps them as the artifact `eeglab-screenshots`).

Manual part: a reviewer checks the committed screenshots in `docs/screenshots/eeglab/` against the checklist in `docs/screenshots/eeglab/README.md` and records date, reviewer and result there. Pass if: - the file dialog shows export 3's `CadLink/Data` folder with its `.ezdataindex` ; - the options dialog shows that file, *Import events* ticked, the concatenating pause choice and the sample-clock timing as defaults, and Help/Cancel/Ok buttons, all readable; - EEGLAB's naming dialog follows, with the recording GUID as the name; - the EEG page shows the 32 channels in the vendor EDF's order (E1/Pg1 ... O2, 1A ... 7A), a 0-10 s axis, Cz flat (the recording reference), one *Øyne lukket* event at 4.6 s, and plausible EEG without import artefacts (steps, clipping, zero blocks, repeated segments).

Repeat the manual part whenever the plugin's dialogs or data path change.

Level	18.0	Parents	DES016 , DES022	Children	-
-------	------	---------	---	----------	---

Traceability matrix

Every chain from need to test; a blank cell means the chain is not linked at that level.

Needs (NEED)	Requirements (REQ)	Design (DES)	Test specifications (TST)
NEED001	REQ001	DES001	TST014
NEED001	REQ006	DES006	TST006
NEED002	REQ002	DES002	TST001
NEED002	REQ005	DES005	TST004
NEED002	REQ005	DES005	TST016
NEED002	REQ020	DES020	TST003
NEED002	REQ021	DES021	TST016
NEED003	REQ003	DES003	TST002
NEED003	REQ003	DES003	TST003
NEED003	REQ004	DES004	TST002
NEED003	REQ005	DES005	TST004
NEED003	REQ005	DES005	TST016
NEED003	REQ007	DES007	TST008
NEED003	REQ008	DES008	TST002
NEED003	REQ008	DES008	TST003
NEED003	REQ008	DES008	TST004
NEED003	REQ008	DES008	TST015
NEED003	REQ009	DES009	TST005
NEED003	REQ010	DES010	TST001
NEED003	REQ018	DES018	TST006
NEED003	REQ019	DES019	TST015
NEED003	REQ020	DES020	TST003
NEED003	REQ021	DES021	TST016
NEED004	REQ011	DES011	TST008
NEED004	REQ012	DES012	TST010
NEED004	REQ013	DES013	TST009
NEED004	REQ017	DES017	TST011
NEED005	REQ004	DES004	TST002
NEED005	REQ007	DES007	TST008
NEED005	REQ014	DES014	TST007
NEED006	REQ006	DES006	TST006
NEED006	REQ016	DES016	TST013
NEED006	REQ016	DES016	TST018
NEED006	REQ022	DES022	TST017
NEED006	REQ022	DES022	TST018
NEED007	REQ015	DES015	TST012